Introduction to Database Systems

2024-Fall

Database & DBMS

- A very large, integrated collection of data.

- Models real-world *enterprise.*
  - Entities (e.g., students, courses)
  - Relationships (e.g., electives)

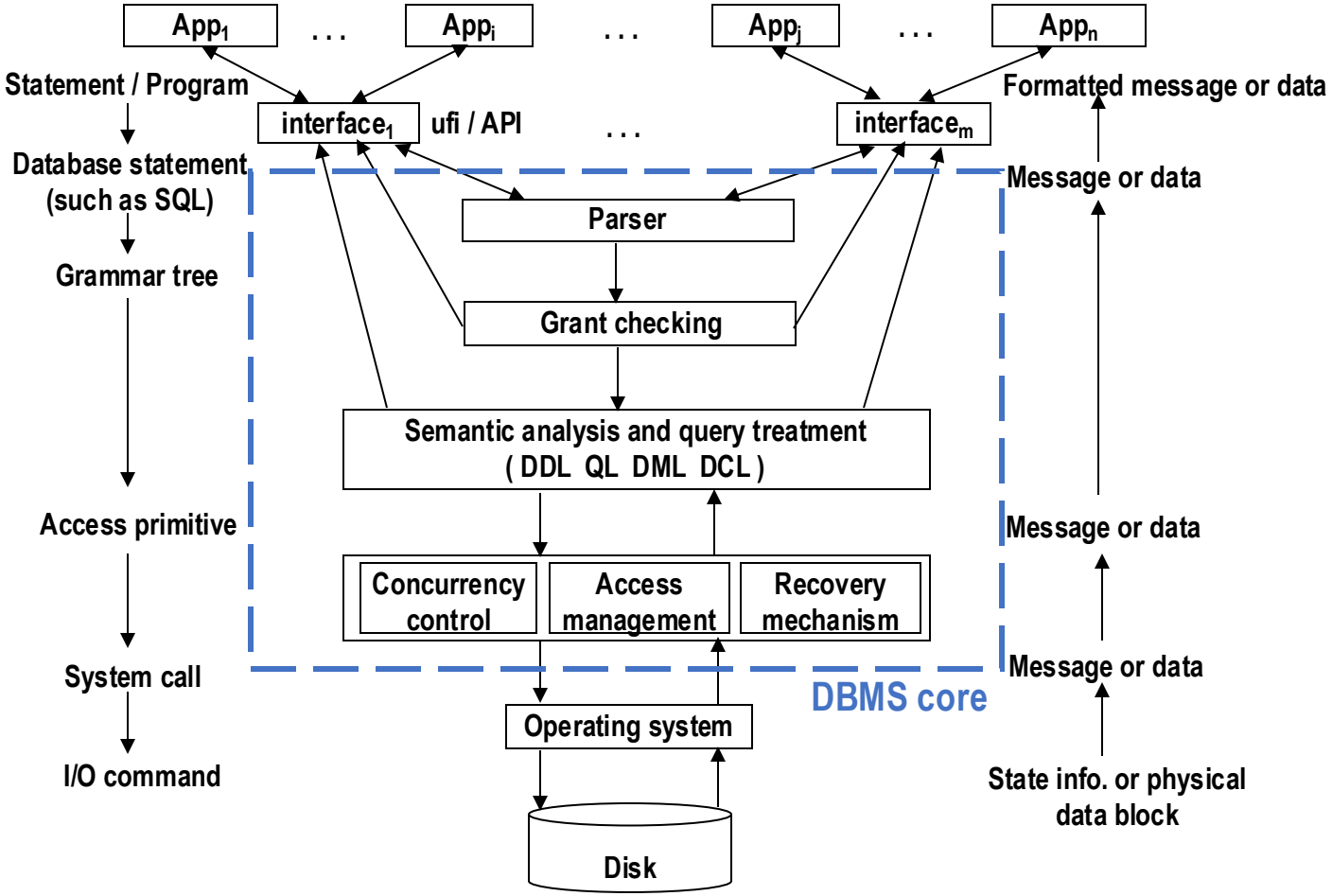
- A *Database Management System (DBMS)* is a software package designed to store and manage databases.

Contents

1. The Architecture of DBMS
   - The components of DBMS core
   - Introduction of transactions
   - The process structure of DBMS
   - Types of DBMS architectures
   - Catalog
2. Database Access Management
3. Query Optimization
4. Transaction Management
   - Concurrent Control
   - Recovery

4.Database Management Systems
（1/5）

The Components of DBMS Core



Transactions

Transaction: Concept and Implementation

- Major component of database systems
- Critical for most applications; arguably more so than SQL

What is a Transaction?

- A *sequence* of *multiple actions* to be executed as an ***atomic*** **unit**
- Application View (SQL View):
  - Begin transaction
  - Sequence of SQL statements
  - End transaction
- Examples
  - Transfer money between accounts
  - Book a flight, a hotel and a car together on Expedia

Transaction Example

- Transaction to transfer $100 from account R to account S

Not seen by the DBMS transaction manager!

1. start transaction
2. read(R)
3. R = R – 100
4. write(R)
5. read(S)
6. S = S + 100
7. write(S)
8. end transaction

ACID: High-Level Properties of Transactions

- **A**tomicity: *All* actions in the **transaction** happen, or *none* happen.
- **C**onsistency: If the DB *starts* out *consistent*, it *ends* up *consistent* at the end of the **transaction**
- **I**solation: Execution of *each* **transaction** is *isolated from* that of *others*
- **D**urability: If a **transaction** *commits*, its effects *persist*.

Note: This is a mnemonic, not a formalism. We'll do some formalisms shortly.

Isolation (Concurrency)

- DBMS interleaves actions of many transactions
  - Actions = reads/writes of DB objects
- DBMS ensures 2 transactions do not “interfere”
- Each transaction executes as if it ran by itself.
  - Concurrent accesses have no effect on transaction’s behavior
  - Net effect must be identical to executing all transactions in some serial order
  - Users & programmers think about transactions in isolation
    - Without considering effects of other concurrent transactions!

Atomicity and Durability

- **A transaction ends in one of two ways:**
  - **Commit** after completing all its actions
    - "commit" is a contract with the caller of the DB
  - **Abort** (or be aborted by the DBMS) after executing some actions
    - Or **system crash** while the transaction is in progress; treat as abort.

- **Two key properties** for a transaction
  - **Atomicity**: Either execute all its actions, or none of them
  - **Durability**: The effects of a committed transaction must survive failures.

- DBMS typically ensures the above by **logging** all actions:
  - **Undo** the actions of aborted/failed transactions.
  - **Redo** actions of committed transactions not yet propagated to disk when system crashes

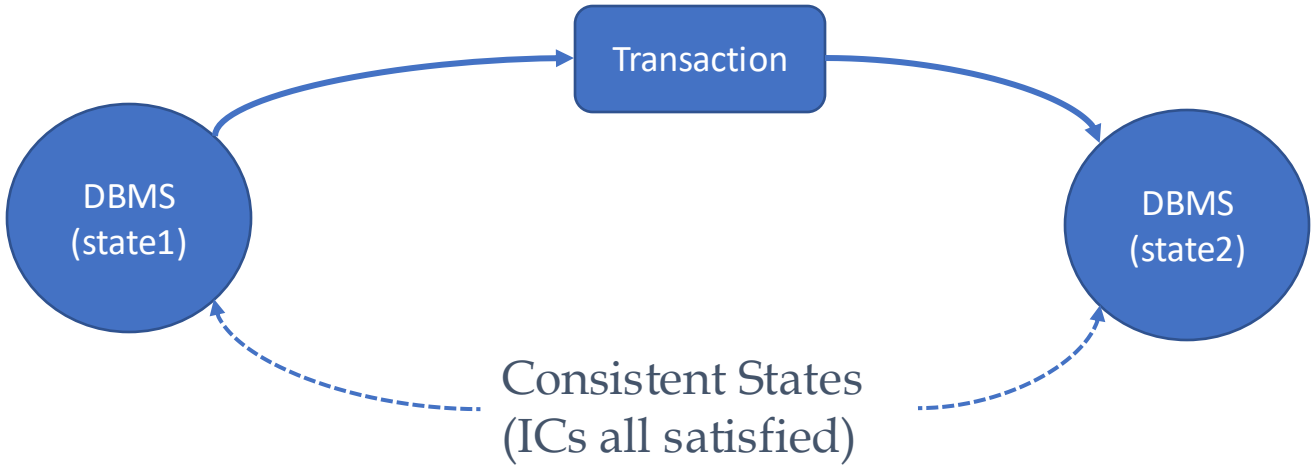
Atomicity and Durability, cont.

- Atomicity
  - If the transaction fails after step 4 and before step 7
    - Money will be "lost" → inconsistent database
  - DBMS should ensure that updates of a partially executed transaction are not reflected
- Durability
  - Once the user hears that the transaction is complete, can rest easy that the $100M was xferred from R to S.

1. start transaction
2. read(R)
3. R = R – 100
4. write(R)
5. read(S)
6. S = S + 100
7. write(S)
8. end transaction

Transaction Consistency

- **Transactions preserve DB consistency**
  - Given a consistent DB state, produce another consistent DB state
- DB consistency expressed as a set of **declarative integrity constraints**
  - CREATE TABLE/ASSERTION statements
- **Transactions that violate integrity are aborted**
  - That's all the DBMS can automatically check!



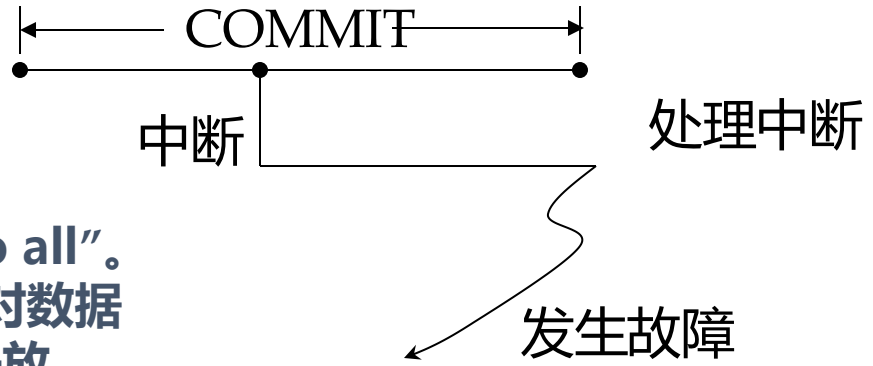
Example

```
BEGIN TRAN
  read A
  A←A–S
  if A < 0 then      /* A款不足*/
             begin
               display “A款不足”
               ROLLBACK  /*出口1*/
             end
           else
             begin
               B←B+S
               display “拨款完成”
               COMMIT    /*出口2*/
             end
```

ROLLBACK 撤销事务的影响，相当于“do nothing”

COMMIT 提交，相当于“do all”。只有在COMMIT之后，事务对数据库产生的变化才对其它事务开放。（为什么？）

- **事务的出口：commit 或 rollback**
- **只有在执行commit之后，事务对数据库所产生的变化才对其他事务开放。**
- **执行commit命令时，要封闭中断，以防处理中断时发生故障**



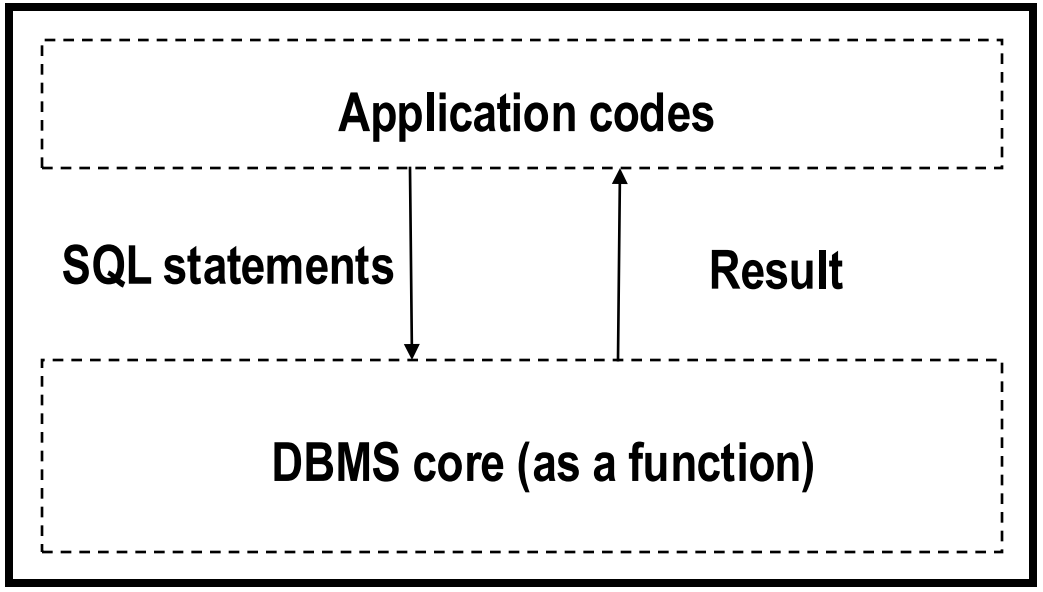
The Process Structure of DBMS

The Process Structure of DBMS

- Single process structure
- Multi processes structure
- Multi threads structure

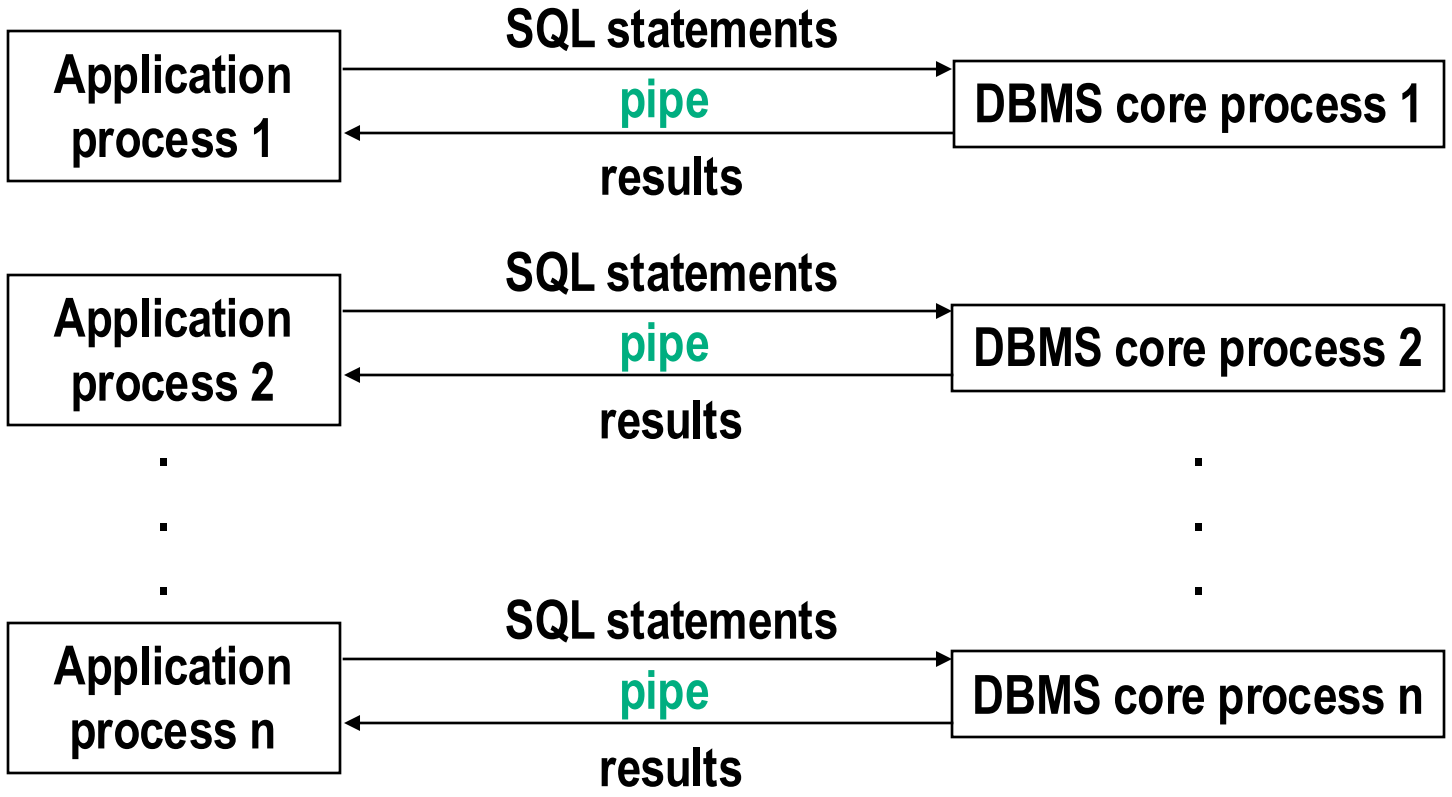
Single process structure

- The application program is compiled with DBMS core as a single .exe file, running as a single process.



Multi processes structure

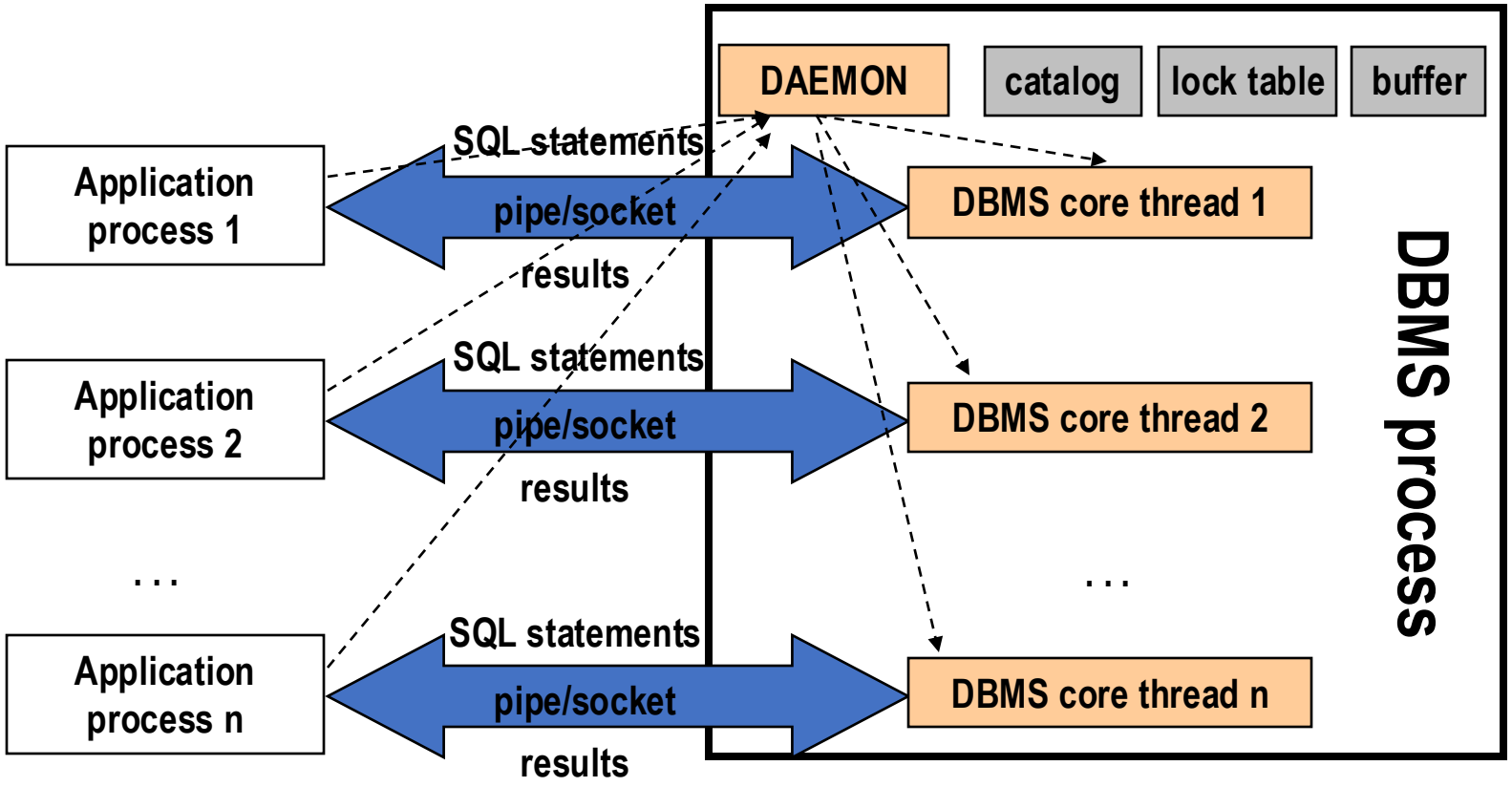
- One application process corresponding to one DBMS core process



优点：实现容易
缺点：
(1).进程的创建、撤销、通信和切换的开销大。
(2).并发事务的增加，进程数激增，内存空间有限，性能下降。
(3).不利于事务共享内存空间。

Multi threads structure

- Only one DBMS process, every application process corresponding to a DBMS core thread.



Multi threads structure

- **线程**是现代OS引入的概念。
  - 以线程为程序并发执行的单位；
  - 一个进程中可创建多个可以相互切换的线程；
  - 这些线程中至少有一个处于就绪状态，进程才处于就绪状态；
  - 进程运行时，其中必有一个线程运行；
  - 同一进程所属的线程共享进程占用的资源，属于线程本身的专用资源很少，描述线程的状态也比进程要少，因此，线程所需资源比进程少；
  - 线程的切换开销和线程间的通信开销小。
- 在多处理机系统中，引入线程，增强了进程的可并发程度。
- 单进程多线程的DBMS中，系统只创建一个DBMS进程（用户接口仍然是进程）。该进程中有常驻的公共服务线程和应用户要求而创建的用户线程。 ——DBMS的并发执行从进程级改为线程级。

Multi threads structure

- 尽管很多现代OS的核心具有线程管理的功能，但对DBMS来说，还是在DBMS进程（相对于OS，是用户进程）中实现线程为宜。理由如下:
  - 可以按照DBMS的需要确定线程调度策略;
  - 线程的切换在用户态，不必转入操作系统的核心态，切换开销小;
  - 可以在不支持线程的操作系统上运行，减少对操作系统的依赖，有利于提高操作系统的可移植性。
- 由DBMS管理线程，需要OS提供如下支持:
  - 提供非阻塞I/O（Nonblocking I/O）和异步I/O（asynchronous I/O）功能;
  - 支持“公平”调度（fair schedule）;
- 即不把具有多线程的DBMS进程，与其它进程等同看待，应区分轻重。

Types of DBMS architectures

Types of DBMS architectures

1.分时系统环境下的集中式数据库系统结构

- 应用的要求以及软硬件条件决定了数据库系统以集中为宜，数据库建立在本单位的主要计算机上，用户通过终端或远距离终端分时访问。
- 数据及其管理都是集中的，数据库系统的所有功能，从用户接口到DBMS核心都集中在DBMS所在的计算机上。

Types of DBMS architectures

2.网络环境下的客户/服务器结构

20世纪70年代：微机的出现和迅速发展；计算机网络的发展和广泛应用，改变了计算机应用系统的格局。

客户机/服务器是一种特殊的分布式处理系统。其中，有一至多台称为客户机的计算机和一至多台称为服务器的计算机通过网络联接。

可以将DBMS的核心部分放在服务器中，而客户机处理数据库的接口部分。客户机也可以有自己的局部DBMS。

客户机面向用户，接受任务，并将任务中需要由服务器完成的部分委托服务器执行。而服务器只接受客户机的委托，完成特定的任务，例如数据库服务。因此，处理是分布的，数据却是集中的，仍属于集中式数据库系统。

Types of DBMS architectures

2.网络环境下的客户/服务器结构

客户器与服务器划分界面的一般原则是：

1）客户提供用户接口、执行应用程序，对服务器提出服务请求；

2）服务器只完成客户器委托的公共服务；

3）服务器与客户器间的数据交换量要尽可能的少

例如，MS SQL Server，Oracle

三层结构：

Types of DBMS architectures

3.物理上分布、逻辑上集中的分布式数据库结构

数据共享和数据集中管理是数据库的主要特征。随着单位规模的扩大和地理上的分散，集中式数据库系统有如下缺点：

- 通信开销大
- 可扩充性差
- 性能差，瓶颈
- 难以管理
- 可用性差

由于存在这些缺点，从20世纪70年代后期，开始了分布式数据库系统的研究。

Types of DBMS architectures

3.物理上分布、逻辑上集中的分布式数据库结构

- 物理上分布、逻辑上集中的分布式数据库结构的思想是：把全局数据模式按数据的来源和用途，合理分布在系统的多个节点上，使大部分的数据可以就近存取。
- 缺点：全局数据模式很难设计、管理、扩充和修改（类似高度集中的计划经济难以管理）。

Types of DBMS architectures

4.物理上分布、逻辑上分布的分布式数据库结构
（事实上，对大范围统一的逻辑几乎不可能）

- 特点：
  （1）节点自治
  （2）没有全局数据模式
- 每个节点看到的数据模式：
  （1）本节点的数据模式
  （2）供本节点共享的其它节点上有关的数据模式
- 没有全局数据模式，节点数据模式的修改甚至节点的加入、撤离，仅仅影响有关的节点。
- 这种分布式数据库系统又称为“联邦式数据库系统”（federated distributed database system）。

Catalog

数据目录

- 数据目录（catalog）存放一组关于数据的数据（描述数据模式的数据），也叫**元数据（meta-data）。**
- DBMS的任务是管理大量的、共享的、持久的数据，有关这些数据的描述需长期保存，一般把这些元数据组成若干表，即数据目录。
- 数据目录既是数据，又不同于一般数据，数据目录也是表，可供查询，主要为DBMS服务，数据目录本身的定义和描述也包含在数据目录中。数据目录只能由系统定义，为系统所有。在初始化时，由系统自动生成（递归初始，类比编译的符号表）
- 数据目录中一般包含下列表：

  SYSTAB、SYSCOL、SYSIDX、SYSVIEW、SYSVWATR

数据目录

- 元数据可以分为2类：
  （1）相对稳定：基表、视图和索引的定义；
  （2）经常变化：数据库状态的统计，例如，元组个数、现有不同属性值的个数等——主要用于查询优化，不必太准，可以定期更新
- 数据目录是影响系统全局的以读为主的数据，对系统的效率影响很大。

Big picture:
Architecture of a DBMS

Architecture of a DBMS: SQL Client

- Last few lectures: SQL
- Next:
  - How is a SQL query executed?

DBMS: Parsing & Optimization

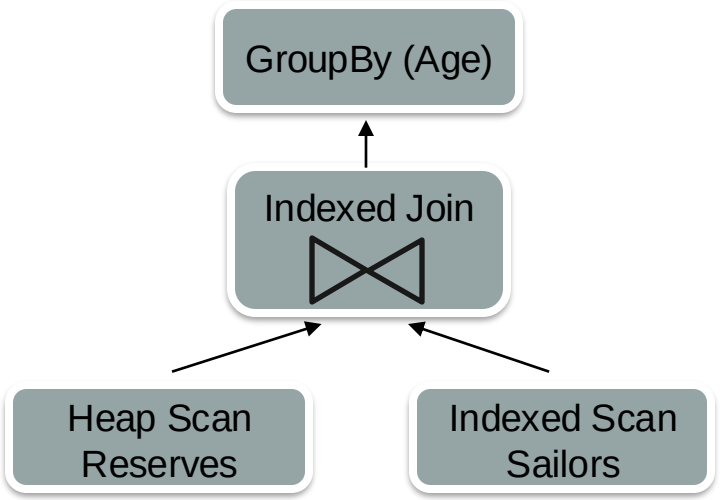
Purpose: Parse, check, and verify the SQL

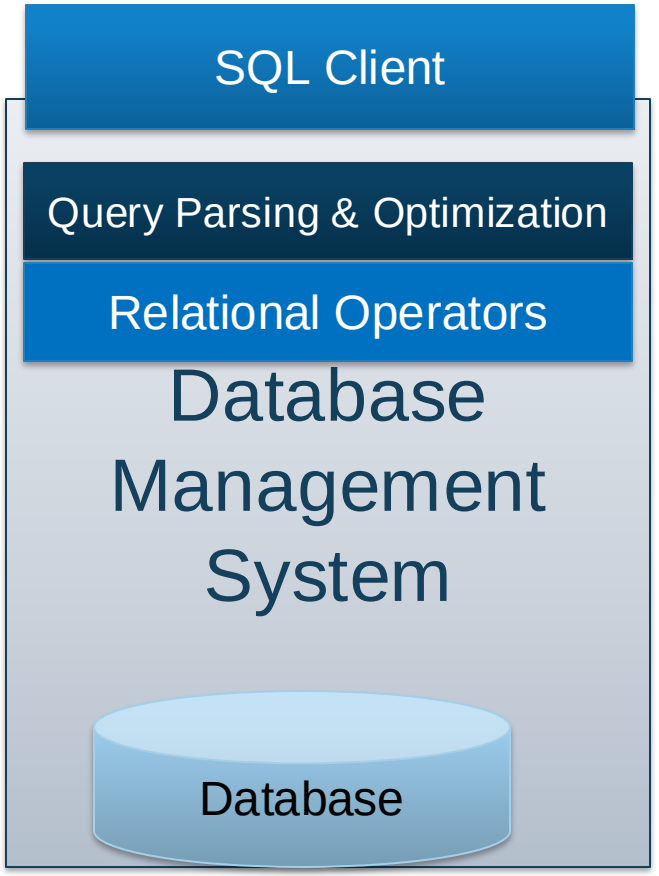
```
SELECT S.sid, S.sname, R.bid
FROM Sailors R, Reserves R
WHERE S.sid = R.sid and S.age > 30
GROUP BY age
```

And translate into an efficient relational query plan.

DBMS: Relational Operators

***Purpose*: Execute a dataflow by operating on **records** and **files**

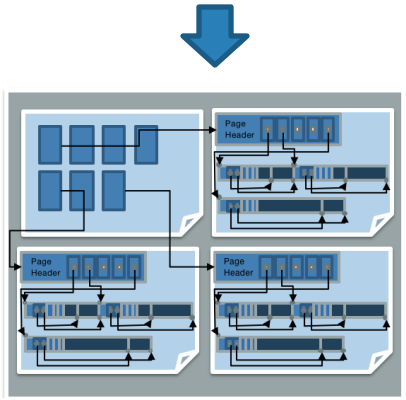


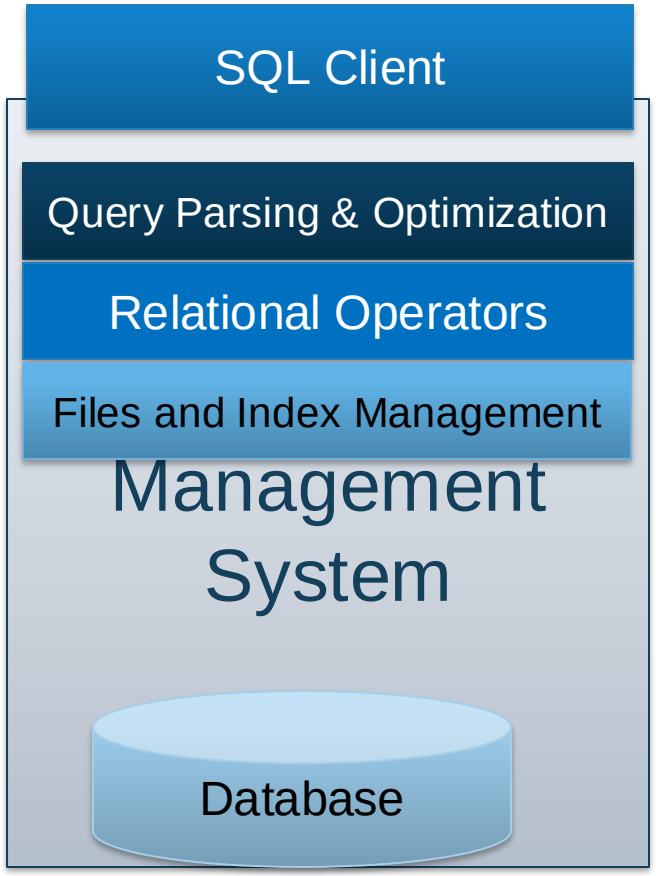


DBMS: Files and Index Management

Purpose: Organize tables and Records as groups of pages in a logical file

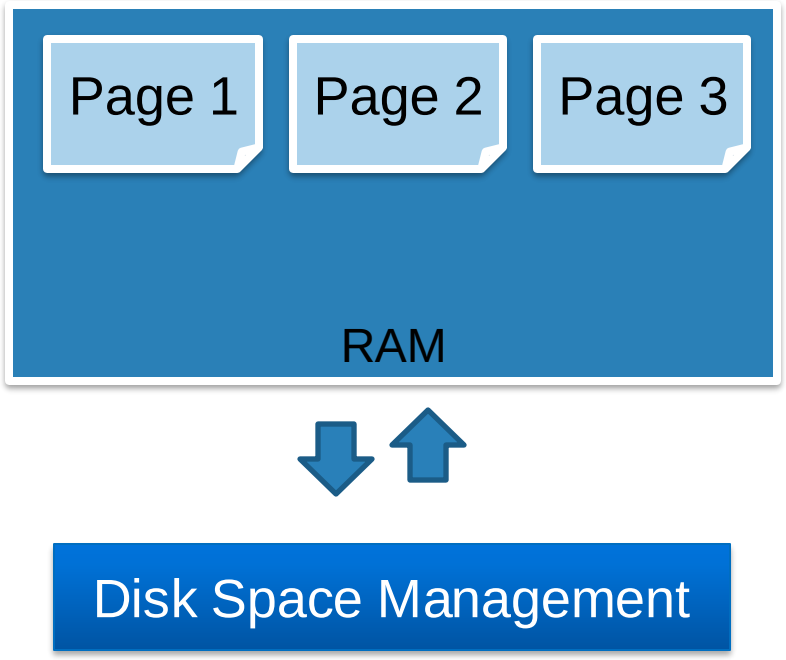
SSN	Last Name	First Name	Age	Salary
123	Adams	Elmo	31	$400
443	Grouch	Oscar	32	$300
244	Oz	Bert	55	$140
134	Sanders	Ernie	55	$400

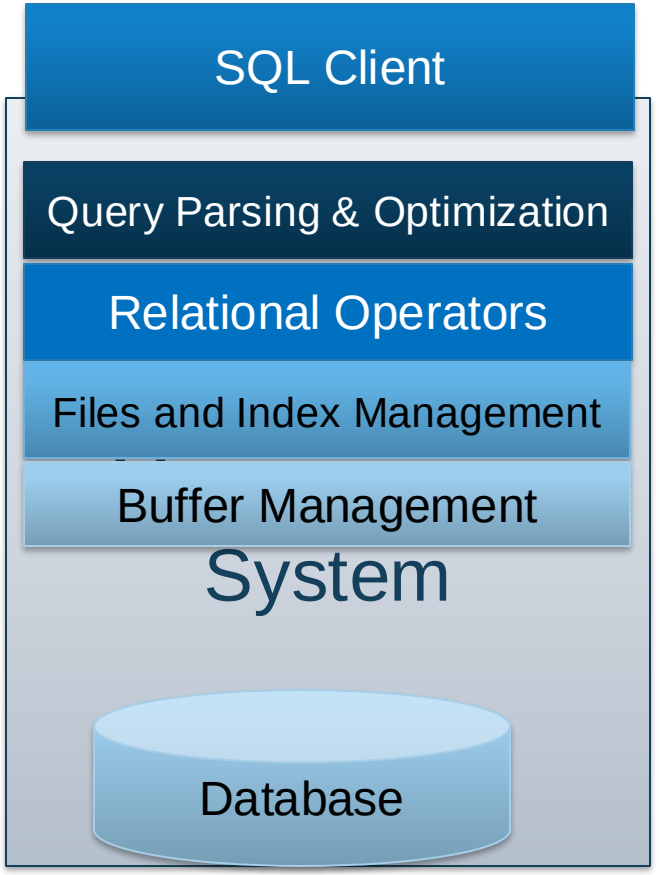




DBMS: Buffer Management

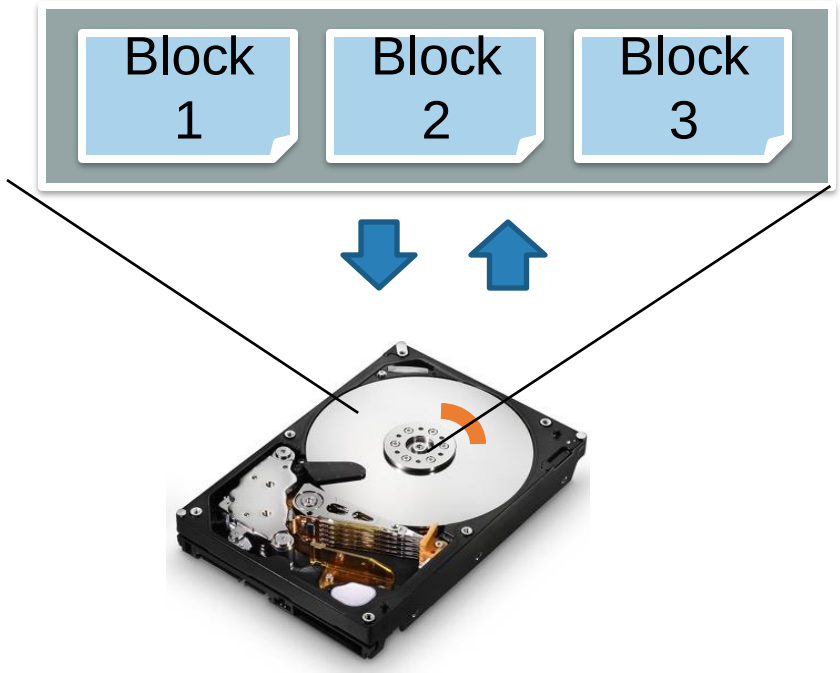
Purpose: Provide the illusion of operating in memory

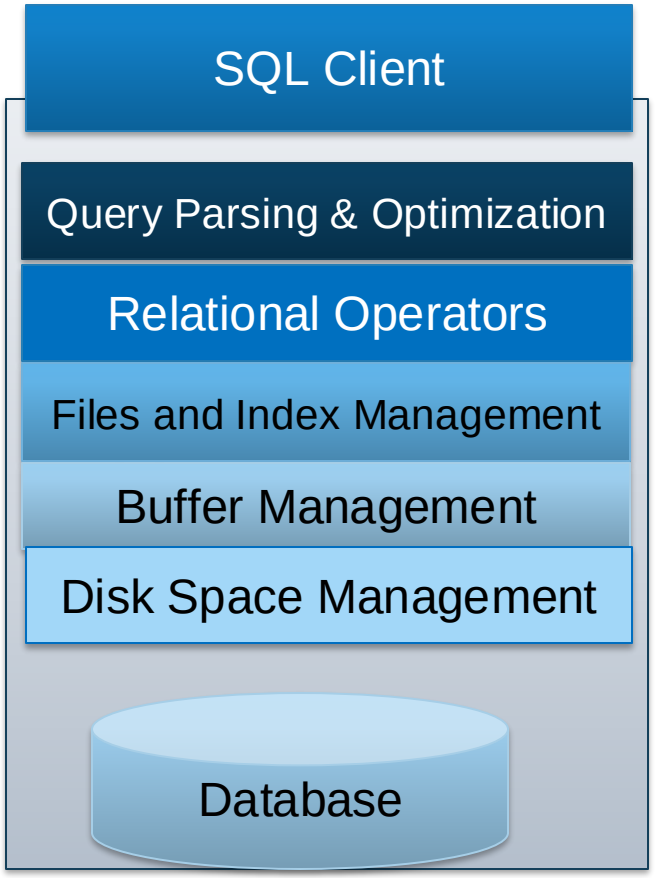




DBMS: Disk Space Management

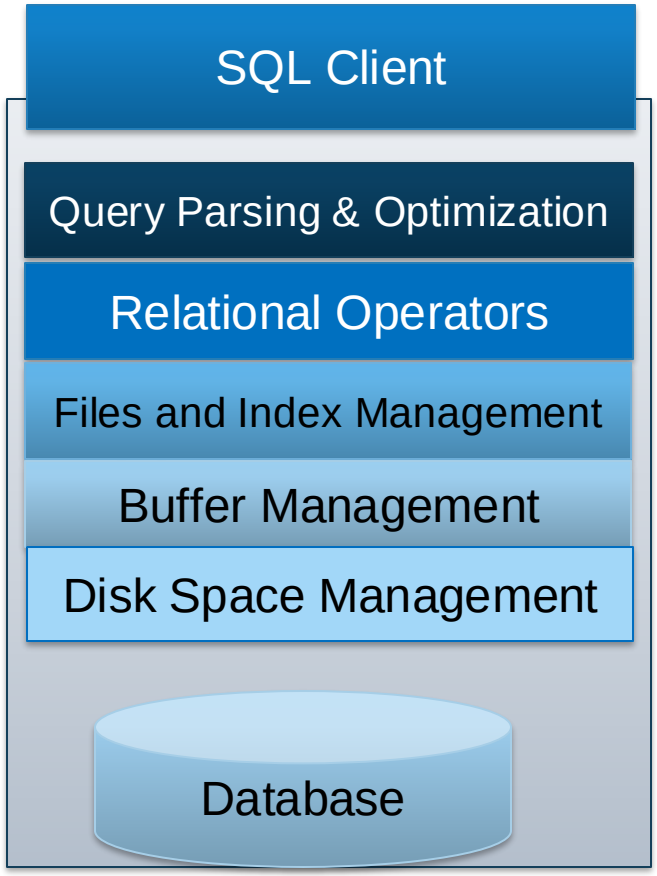
***Purpose*:** Translate page requests into physical bytes on one or more device(s)





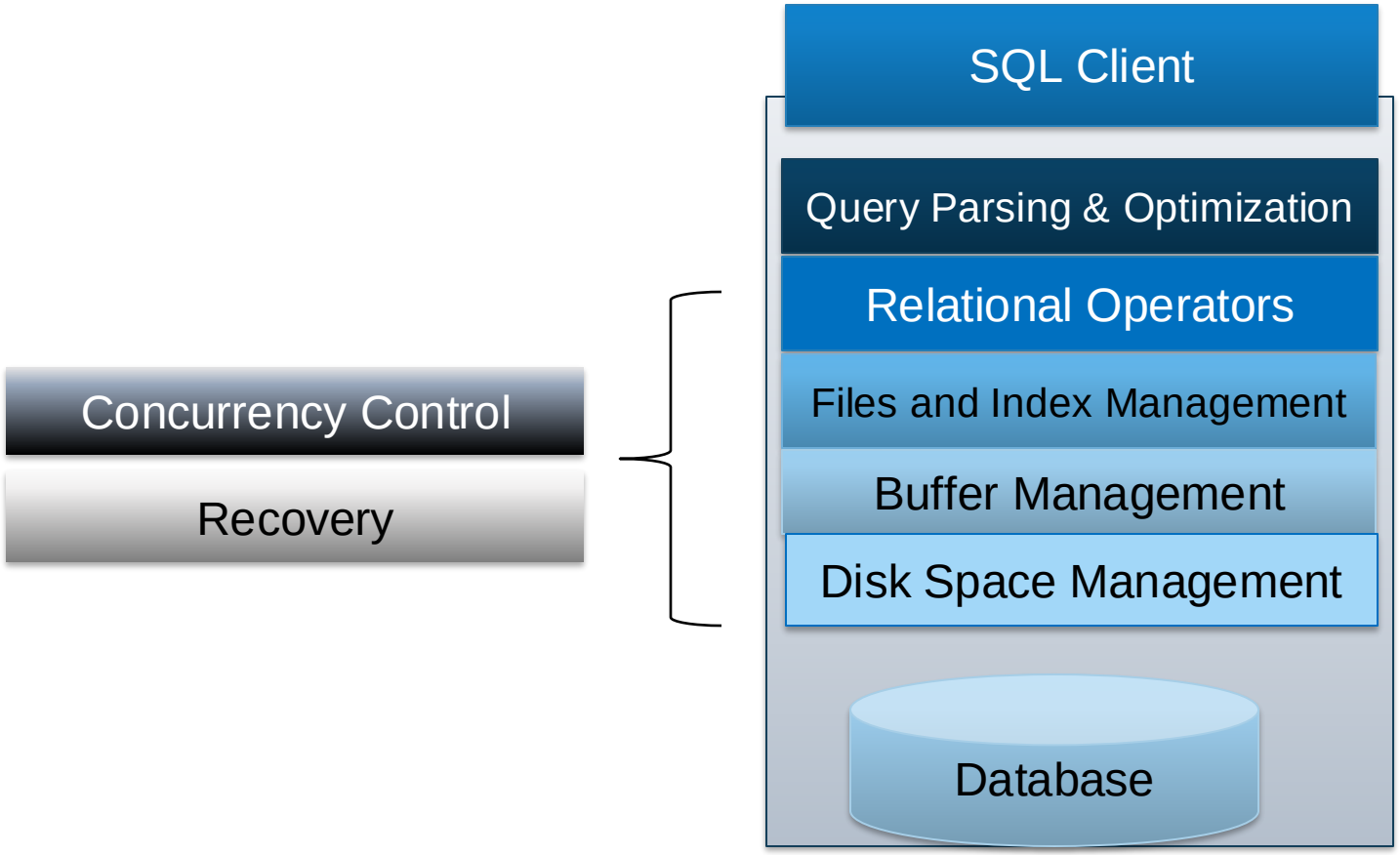
Architecture of a DBMS

- Organized in layers
- Each layer abstracts the layer below
  - Manage complexity
  - Performance assumptions
- Example of good systems design



DBMS: Concurrency & Recovery

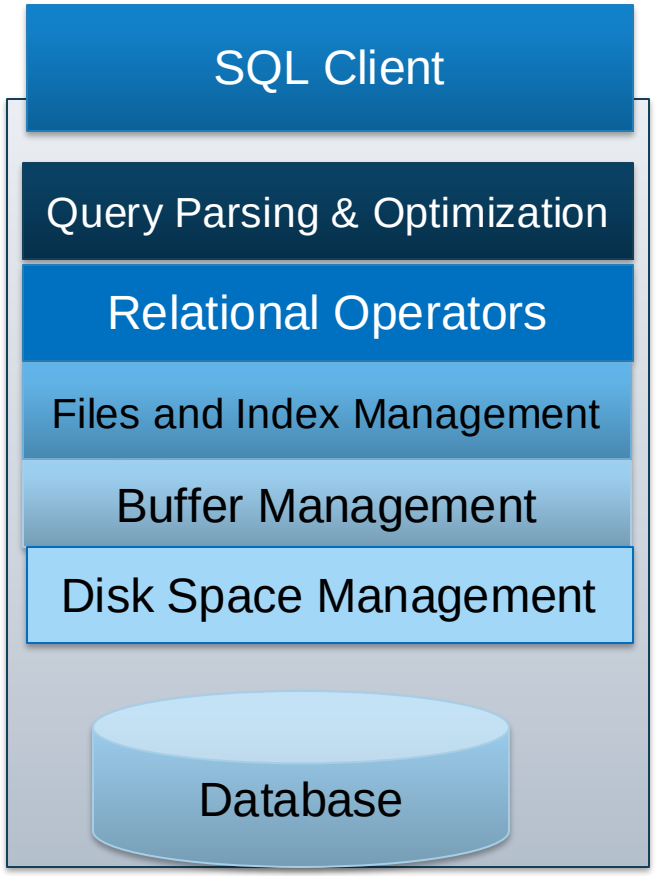
Two cross-cutting issues related to storage and memory management:



Context

Completed

You are Here

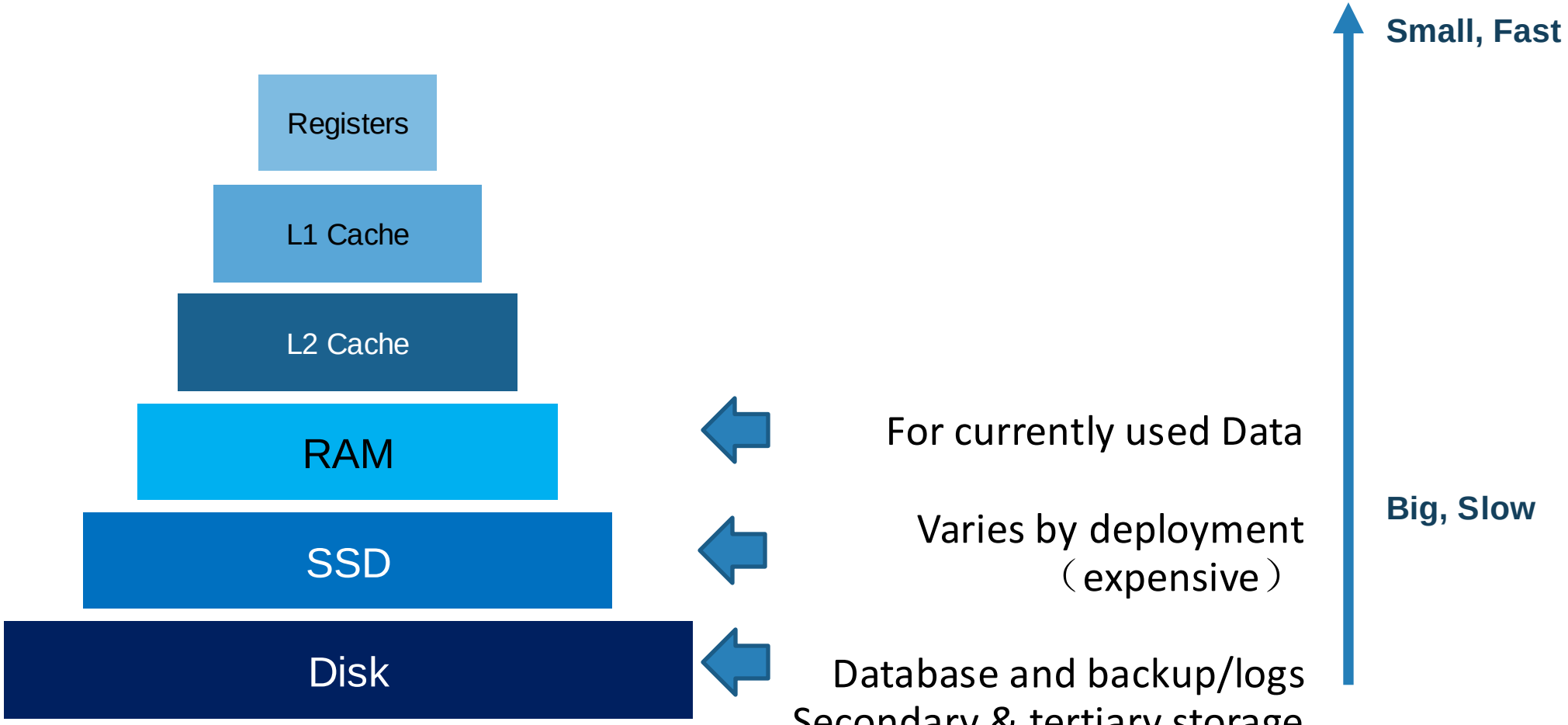


Before We Begin:
Storage Media

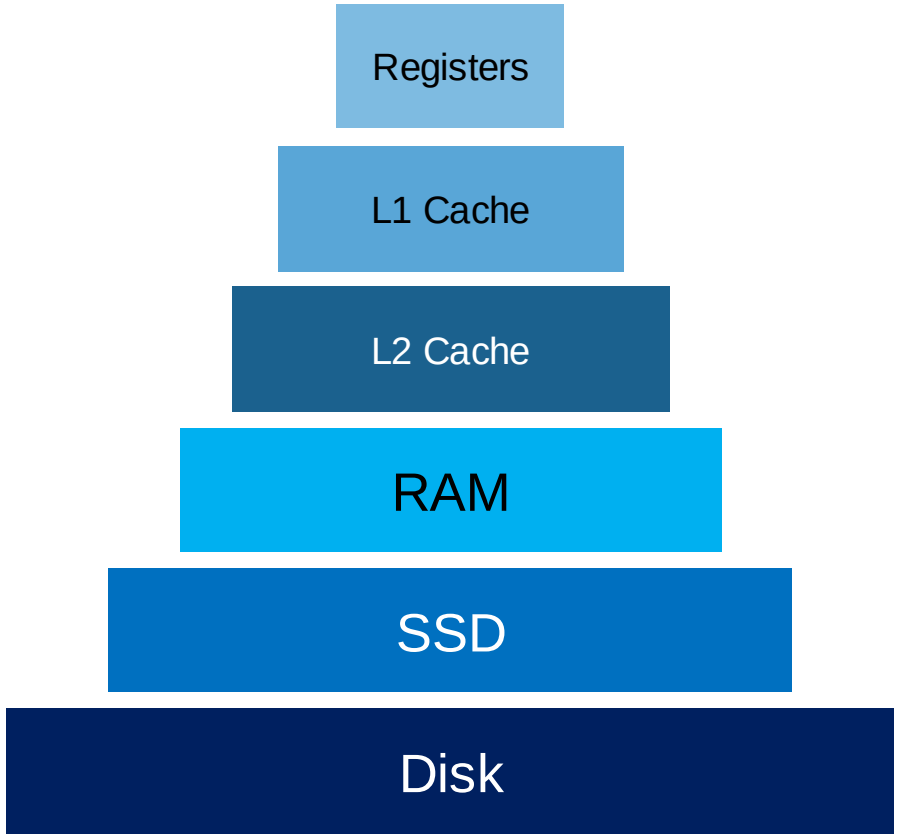
Disks

- Can differentiate storage into:
  - **volatile storage**: loses contents when power is switched off
  - **non-volatile storage**:
    - Contents persist even when power is switched off.
    - Includes secondary and tertiary storage, as well as batter-backed up main-memory.
- Factors affecting choice of storage media include
  - Speed with which data can be accessed
  - Cost per unit of data
  - Reliability

Storage Hierarchy



Hierarchy - Storage Latencies



.5 ns – L1 cache reference

7.0 ns – L2 cache reference

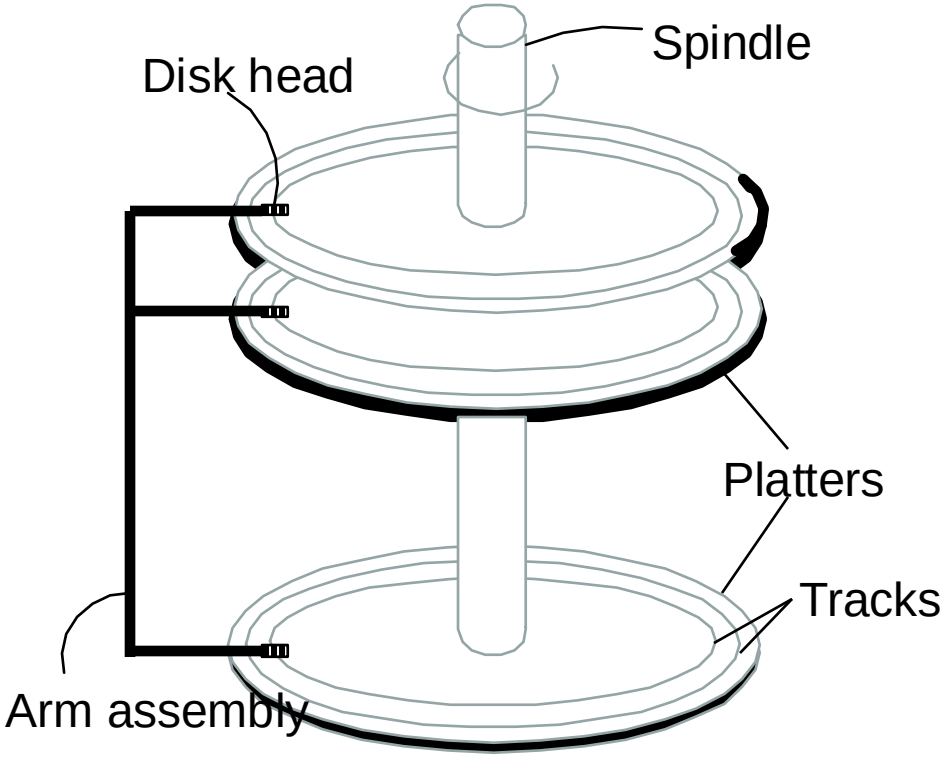
100.0 ns – main memory reference

1,000,000.0 ns – to read 1MB sequentially

20,000,000.0 ns to read 1MB sequentially

Components of a Disk, Pt. 1

- **Platters** spin (say 15000 rpm)
- **Arm assembly** moved in or out to position a **head** on a desired **track**
  - Tracks under heads make a “cylinder”
- Only one head reads/writes at any one time

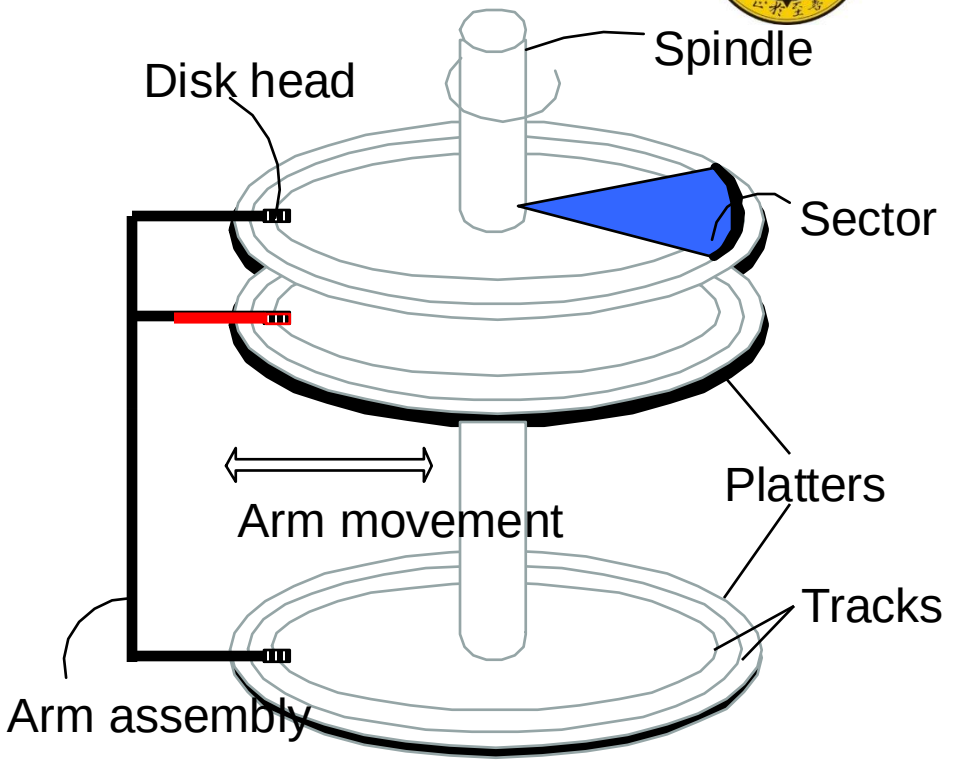


Disk Platters

活动头磁盘的存取时间由三部分组成：寻道时间、等待时间以及传输时间

Components of a Disk, Pt. 2

- **Platters** spin (say 15000 rpm)
- **Arm assembly** moved in or out to position a **head** on a desired **track**
  - Tracks under heads make a “cylinder”
- Only one head reads/writes at any one time
- Block/page size is a multiple of (fixed) **sector** size



磁盘上的数据划分为大小相等的物理块。磁盘与内存间的数据交换以物理块为单位。

以物理块为交换单位的优点：

1). 减少I/O的次数，从而减少寻道和等待的时间。

2). 减少间隙的数目，提高磁盘空间利用率。

物理快的大小由OS决定。

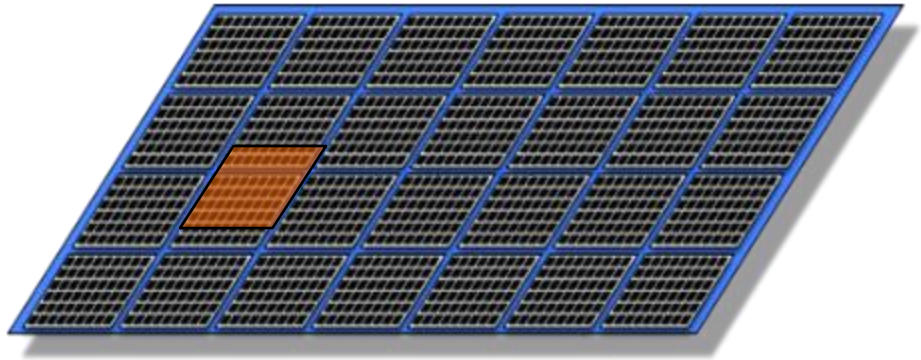
Accessing a Disk page

- Time to access (read/write) a disk block:
  - **seek time** (moving arms to position disk head on track)
    - ~2-3 ms on average
  - **rotational delay** (waiting for block to rotate under head)
    - ~0-4 ms (15000 RPM)
  - **transfer time** (actually moving data to/from disk surface)
    - ~0.25 ms per 64KB page

- Key to lower I/O cost: ***reduce seek/rotational delays***

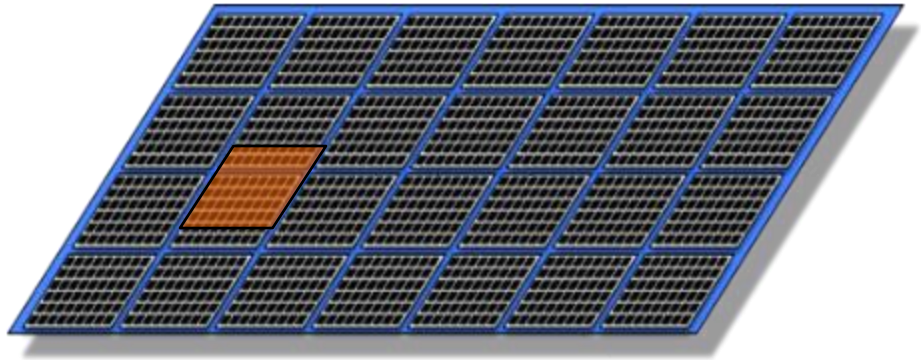
Notes on Flash (SSD)

- Issues in current generation (NAND)
  - Fine-grain reads (4-8K reads), coarse-grain writes (1-2 MB writes)
  - Only 2k-3k erasures before failure,
  - Write amplification: big units, need to reorg for wear & garbage collection



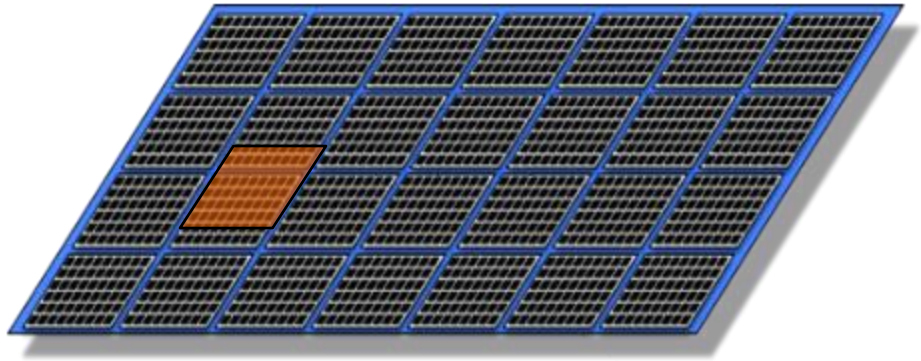
Notes on Flash (SSD), Pt. 2

- So… read is fast and predictable
  - Single read access time: 0.03 ms
  - 4KB random reads: ~500MB/sec
  - Sequential reads: ~525MB/sec
  - 64K: 0.48 ms



Notes on Flash (SSD), cont

- But... write is not! Slower for random
  - Single write access time: 0.03 ms
  - 4KB random writes: ~120 MB/sec
  - Sequential writes: ~480 MB/sec

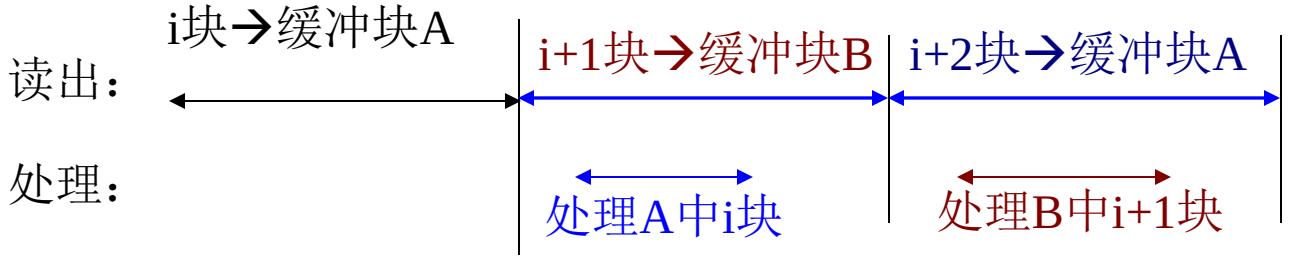


Bottom Line (last few years)

- Very large DBs: relatively traditional
  - Disk still the best cost/MB by a lot
  - SSDs improve performance and performance variance
- Smaller DB story is changing quickly
  - Entry cost for disk is not cheap, so flash wins at the low end
  - Many interesting databases fit in RAM

Buffer Management

- In general, a buffer is established between the disk and memory to address the speed mismatch between the two.
- 由于有多个缓冲块可供申请使用，磁盘的读写操作和读写数据的处理可以重叠进行。



- Both the OS and DBMS have their own buffers.
- Many DBMSs use techniques such as ***delayed writing*** and ***prefetching*** to reduce I/O and improve performance.

Disk Space Management

Block Level Storage

- Read and Write **large chunks of sequential bytes**
- *Sequentially*: “Next” disk block is fastest
- Maximize usage of data per Read/Write
  - “Amortize” seek delays (HDDs) and writes (SSDs):
- Predict future behavior
  - Cache popular blocks
  - Pre-fetch likely-to-be-accessed blocks
  - Buffer writes to sequential blocks
  - More on these as we go

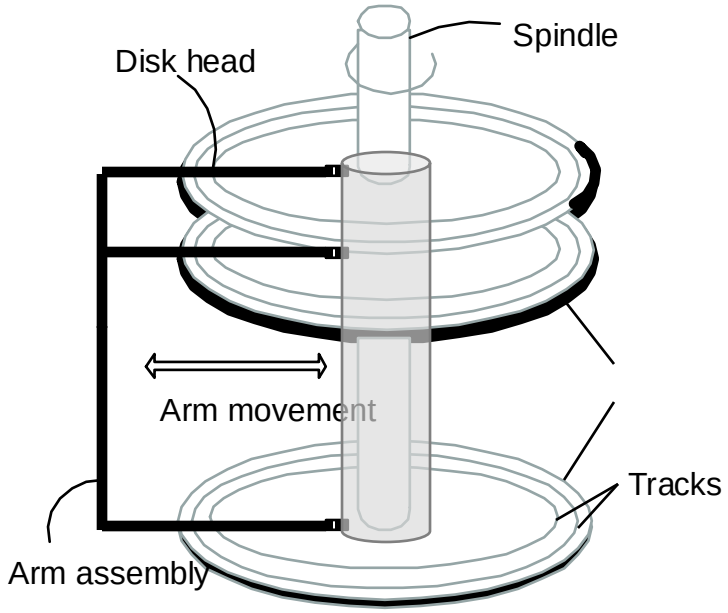
A Note on Terminology

- **Block = Unit of transfer for disk read/write**
  - 64KB – 128KB is a good number today
  - Book says 4KB
- **Page: a common synonym for “block”**
  - In some texts, “page” = a block-sized chunk of RAM

- We’ll treat “block” and “page” as synonyms

Arranging Blocks on Disk

- **'Next'** block concept:
  - sequential blocks on same track, followed by blocks on same cylinder, followed by blocks on adjacent cylinder
- Arrange file pages sequentially by 'next' on disk
  - minimize seek and rotational delay.
- For a **sequential scan**, *pre-fetch*
  - several blocks at a time!
- **Read large consecutive blocks**



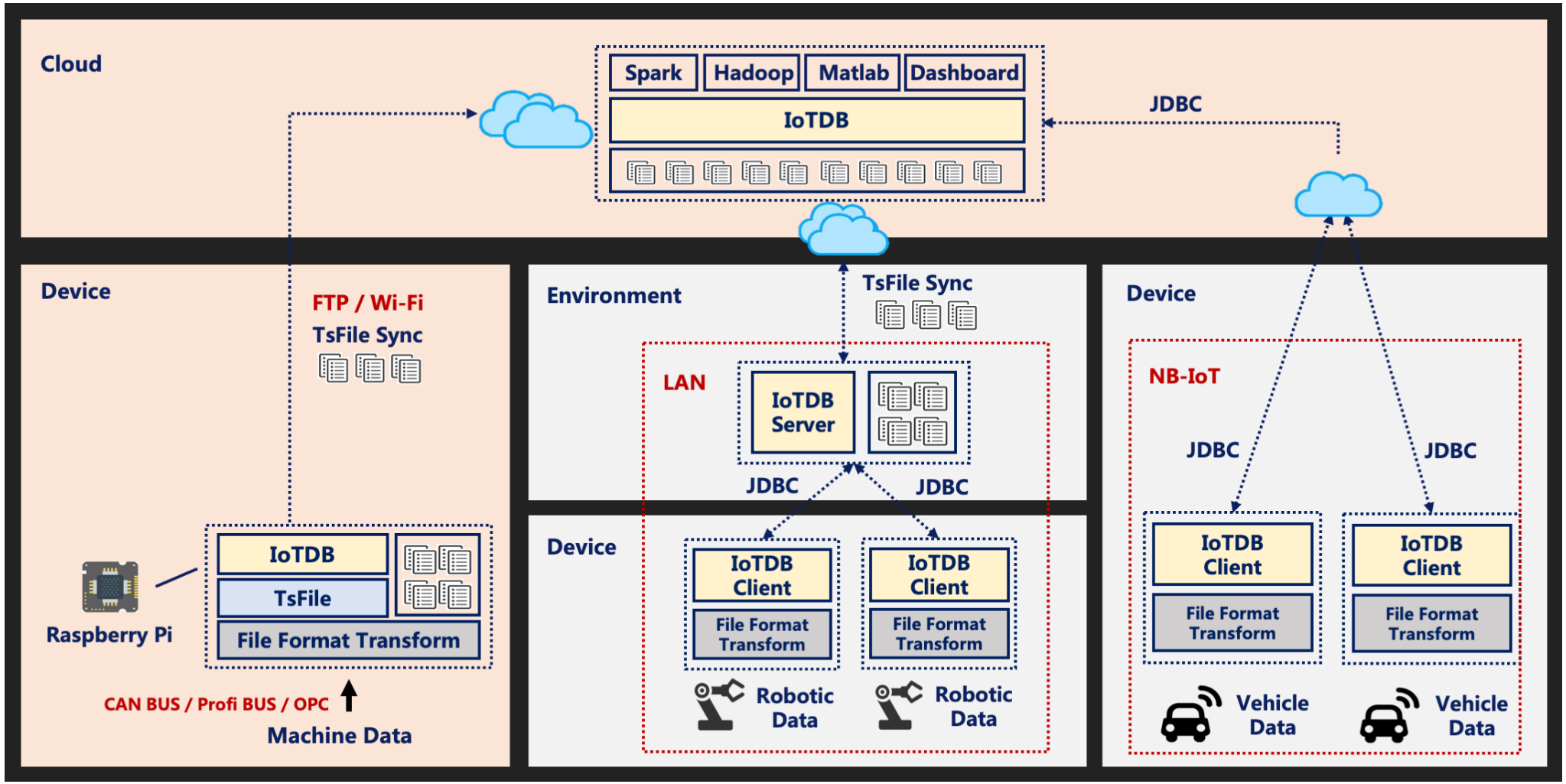
Disk Space Management: Implementation

- **Proposal 1:** Talk to the storage device directly
  - Could be very fast if you knew the device well
  - What happens when devices change?

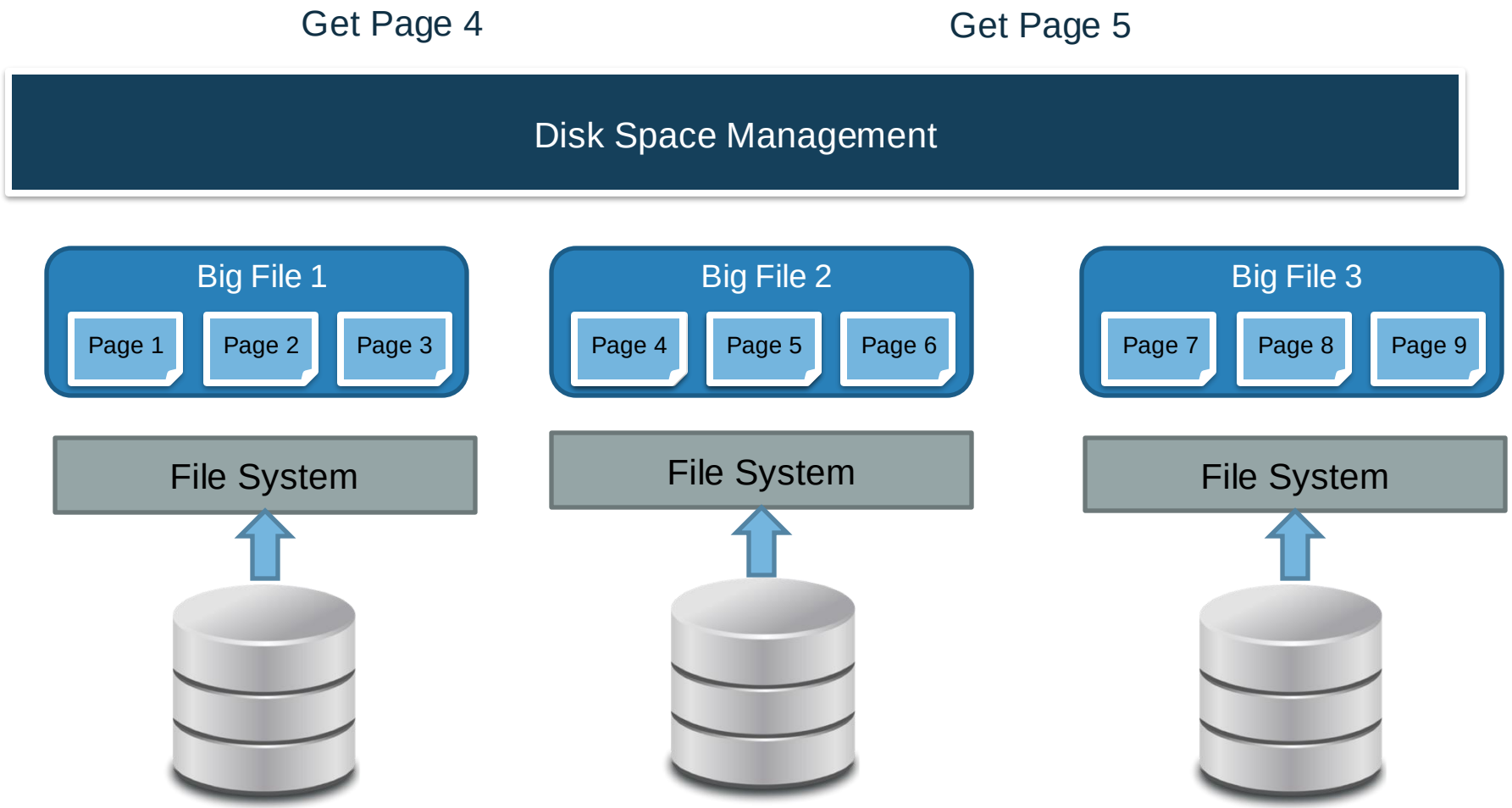
Disk Space Management: Implementation 2

- **Proposal 2**: Run over filesystem (FS)
  - Allocate single large “contiguous” file on a nice empty disk, and assume sequential/nearby byte access are fast
  - Most FS optimize disk layout for sequential access
    - Gives us more or less what we want if we start with an empty disk
  - DBMS “file” may span multiple FS files on multiple disks/machines

Example: IoTDB

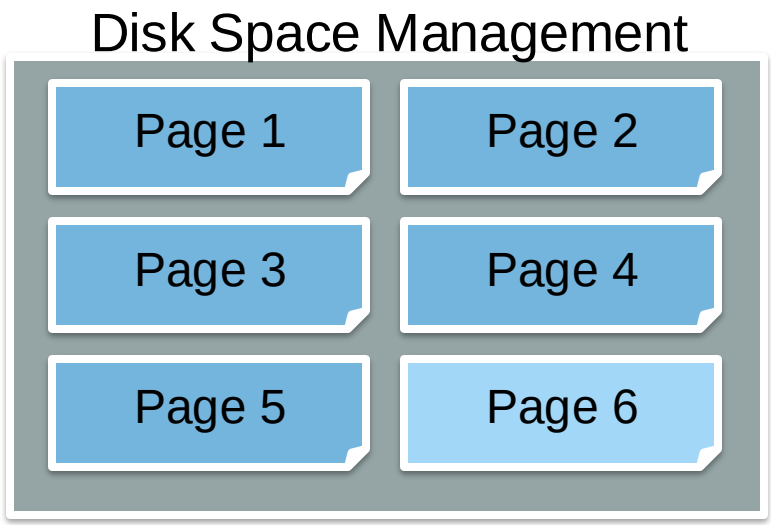


Using Local Filesystem



Summary: Disk Space Management

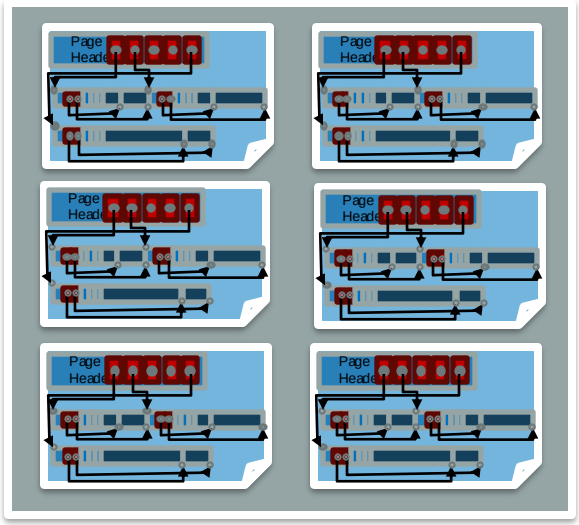
- Provide API to read and write pages to device
- Pages: block level organization of bytes on disk
- Provides “next” locality and abstracts FS/device details



4.Database Management Systems
（2/5）

Overview: Files of Pages of Records

- Tables stored as logical files
  - Consist of pages
    - Pages contain a collection of records
- Pages are managed
  - On disk by the disk space manager:
    pages read/written to physical disk/files
  - In memory by the buffer manager:
    higher levels of DBMS only operate in memory



Records

Record Formats

- Relational Model
  - Each record in table has some fixed type
- Assume System Catalog stores the Schema
  - No need to store type information with records (save space!)
  - Catalog is just another table …
- Goals:
  - Records should be compact in memory & disk format
  - Fast access to fields (why?)

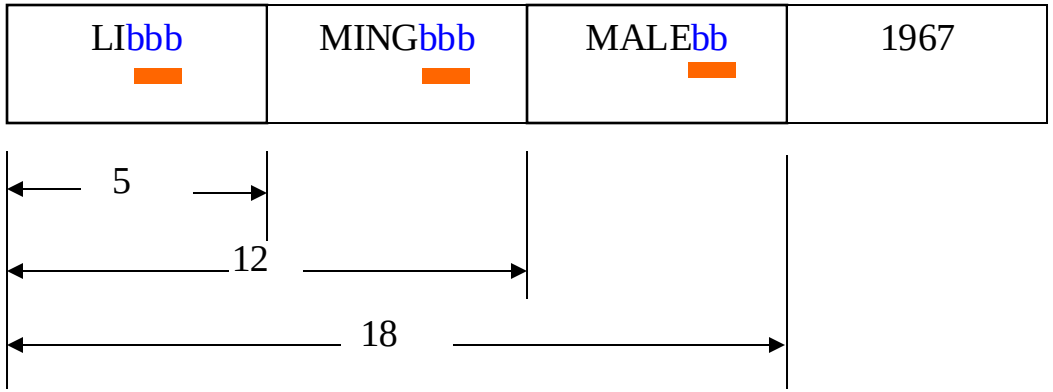
- Easy Case: Fixed Length Fields
- Interesting Case: Variable Length Fields

Fixed-Length Records

- Simple approach:
  - Store record *i* starting from byte *n* ∗ *(i* – 1), where *n* is the size of each record.
  - Record access is simple but records may cross blocks
    - *Modification: do not allow records to cross block boundaries*

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

字段-定位法



Fixed-Length Records

- Deletion of record *i:* alternatives*:*
  - **move records *i* + 1, . . ., *n* to *i*, . . . , *n* – 1**

Record 3 deleted

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Fixed-Length Records

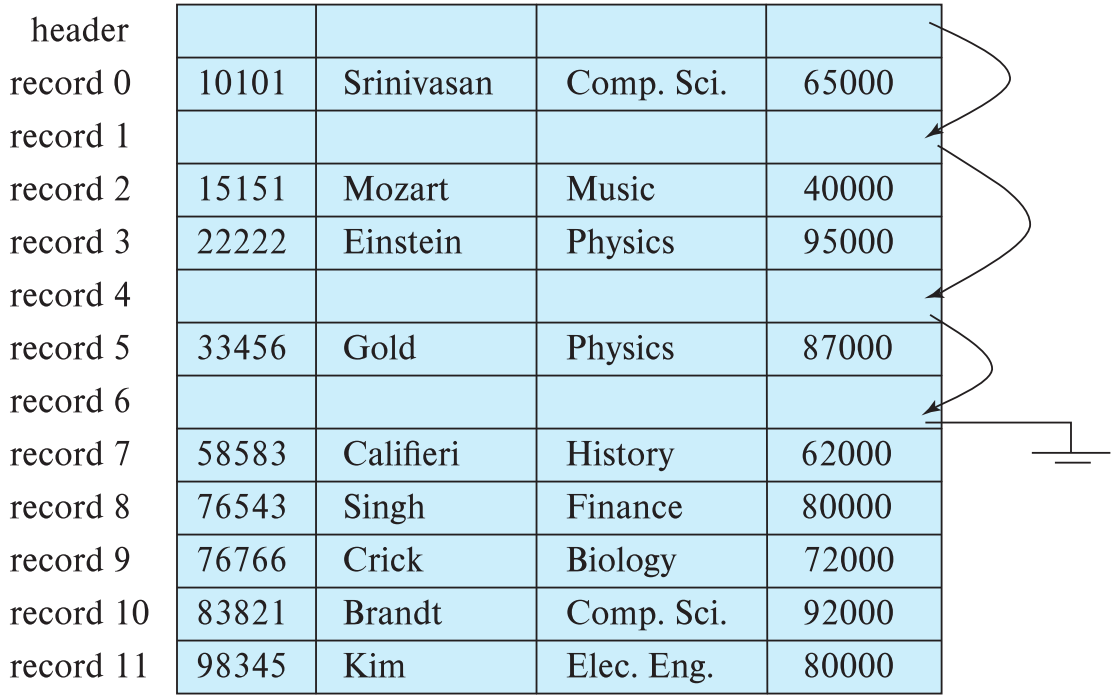
- Deletion of record *i:* alternatives*:*
  - move records *i* + 1, . . ., *n* to *i*, . . . , *n* – 1
  - **move record *n* to *i***

Record 3 deleted and replaced by record 11

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 11	98345	Kim	Elec. Eng.	80000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000

Fixed-Length Records

- Deletion of record *i:* alternatives*:*
  - move records *i* + 1, . . ., *n* to *i*, . . . , *n* – 1
  - move record *n* to *i*
  - **do not move records, but link all free records on a *free list***

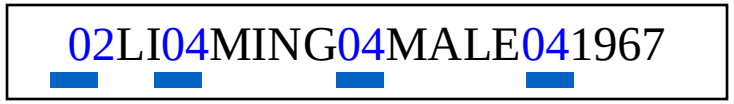


header				
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1				
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4				
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Variable-Length Records

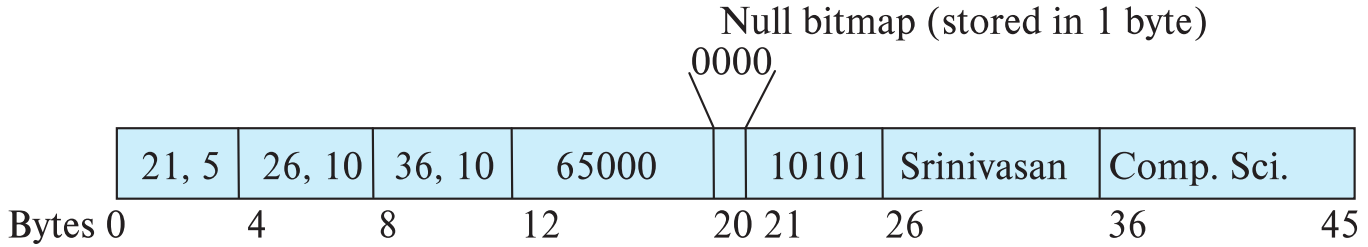
- 相对法——每个字段没有固定的长度，而是用特殊的字符分隔开

- 计数法——每个字段的开始加上表示该字段长度的字段



Variable-Length Records*

- Variable-length records arise in database systems in several ways:
  - Storage of multiple record types in a file.
  - Record types that allow variable lengths for one or more fields such as strings (**varchar**)
  - Record types that allow repeating fields (used in some older data models).
- Attributes are stored in order
- Variable length attributes represented by fixed size (offset, length), with actual data stored after all fixed length attributes
- Null values represented by null-value bitmap



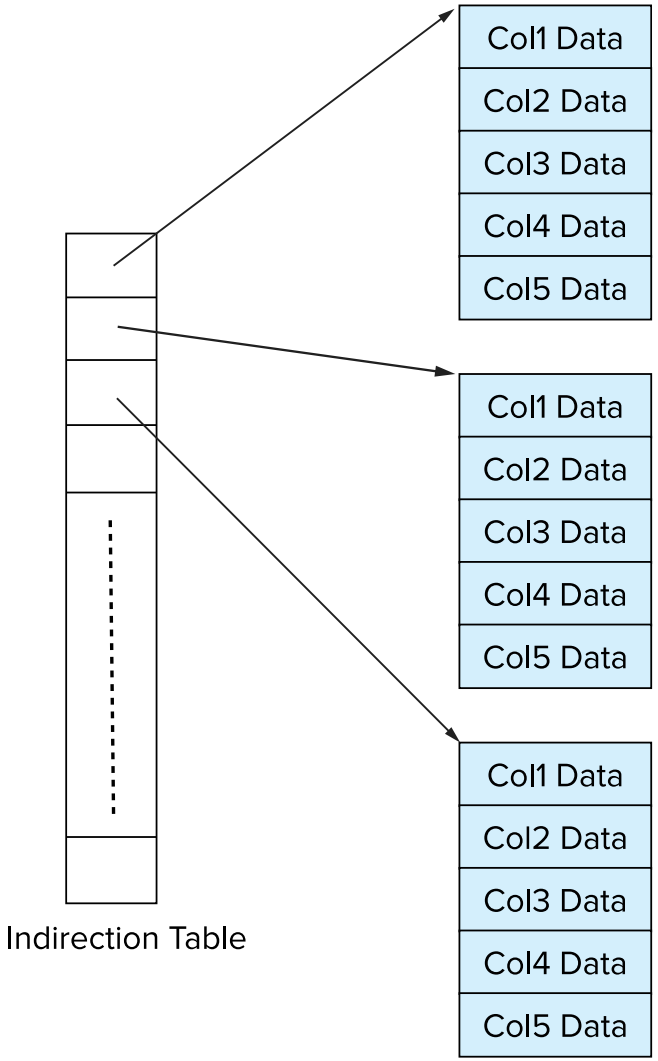
Column-Oriented Storage

- Also known as **columnar representation**
- Store each attribute of a relation separately
- Example

10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Storage Organization in Main-Memory Databases

- Can store records directly in memory without a buffer manager
- Column-oriented storage can be used in-memory for decision support applications
  - Compression reduces memory requirement



Allocation of records in blocks

- Records may cross blocks
  - Unspanned organization (不跨块组织)
  - Spanned organization (跨块组织)
- Given that B is the size of space in a physical block and R is the size of a fixed-length record, if B>R, then the number of records that can be contained in each physical block is:

$$p=\lfloor B/R \rfloor$$

p is the Blocking factor (块因子)

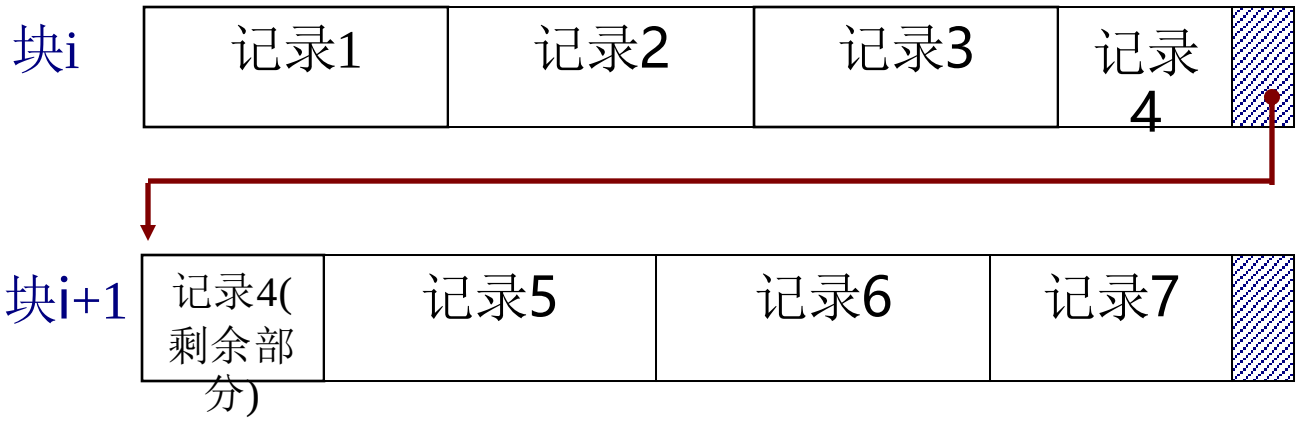
Allocation of records in blocks

- Records may cross blocks
  - Unspanned organization (不跨块组织)
  - Spanned organization (跨块组织)

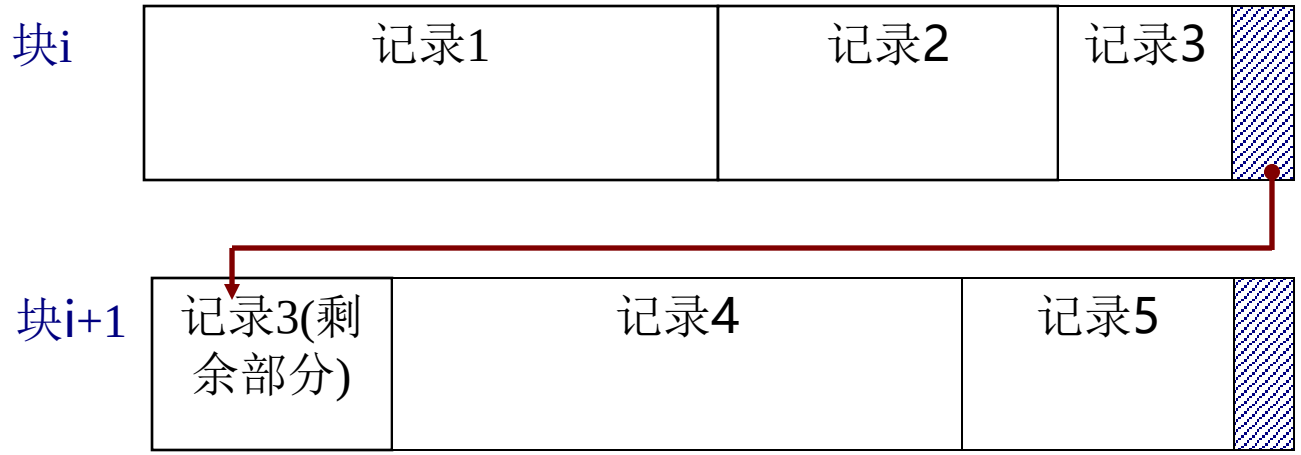
- Records generally do not completely fill the physical block, leaving some unused leftover space.

$$\mathrm{B} - p \times R \ < R$$

Allocation of records in blocks



定长记录（跨块）



变长记录（跨块）

Allocation of blocks in Disks

- In early DBMS, the physical blocks required by the database were typically allocated by the operating system, which could result in logically adjacent data being scattered across different areas of the disk. This led to decreased performance when accessing data.
- In modern DBMS, the required storage space is requested in one go from the operating system during the initialization of the DBMS.

Allocation of blocks in Disks

- contiguous allocation （连续分配法）
  - Allocating a file's blocks in contiguous space on the disk means that the ***blocks are stored in sequential order***, which is beneficial for sequential access to multiple blocks of a file. However, it is not advantageous for expanding the file.
- linked allocation （链接分配法）
  - Physical blocks are not necessarily allocated in contiguous storage space on the disk; instead, each physical block is ***linked using pointers***. This approach is advantageous for file expansion but tends to be less efficient.

Allocation of blocks in Disks

- clustered allocation （簇集分配法）
  - The combination of contiguous allocation and linked allocation
  - Clustered allocation in a DBMS refers to a technique where related data is stored together in close physical proximity on the storage medium, often within the same or adjacent disk blocks.
- indexed allocation （索引分配法）
  - This method utilizes an ***index block*** that contains pointers to the actual data blocks （逻辑块号到物理块地址的索引）, providing a way to efficiently manage non-contiguous storage space.

Data compression （数据压缩技术）

1.消零或空格符法（null suppression）

例如，bbbbb可以用#5表示；

000000可以用@6表示等。

2.串型代替法（pattern substitution）

对反复出现的字符串，可以用一个省略符代替。

原始数据
IBM PC/XT 00001
IBM PC/XT 00002

压缩数据
@#1
@#2

例如，串型表如右：

IBM PC/XT	@
0000	#

Data compression （数据压缩技术）

3.索引法（indexing）

串行代替法的变种，对重复出现的串行，单独存储，在用到这些串行的地方，用指针引用它。

原始数据

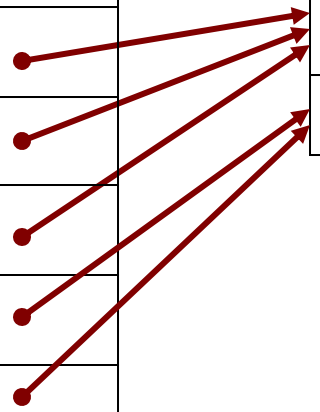
SHOP#	CITY
0001	Nanjing
0002	Nanjing
0003	Nanjing
0004	Shanghai
0005	Shanghai

压缩数据

SHOP#	CITY
0001	
0002	
0003	
0004	
0005	

CITY表

Beijing
Nanjing
Shanghai



Database Files

4.2 Database Access Management

The access to database is transferred to the operations on files (of OS) eventually. The ***file structure*** and ***access route*** offered on it will affect the speed of data access directly. It is impossible that one kind of file structure will be effective for all kinds of data access.

- Access types
- File organization
- Index technique

Access Types

- Query all or most records of a file (>15%)
- Query some special record
- Query some records (<15%)
- Scope query
- Update

Requirements of DBMS

- Some DBMSs use the OS's file management system as the foundation for their physical layer. However, more DBMSs do not rely on the OS's file management system and instead independently design their storage structures. The reasons for this are as follows:
  - Traditional file systems cannot provide the additional information needed to implement DBMS functionality. To achieve its functions, a DBMS must append some information to parts such as file directories, file description blocks, and physical blocks.
  - Traditional file systems are primarily oriented towards ***batch processing***, while database systems require ***instant access and dynamic modification***. This necessitates that the file structure be adaptable to dynamic changes in data and provide fast access paths.

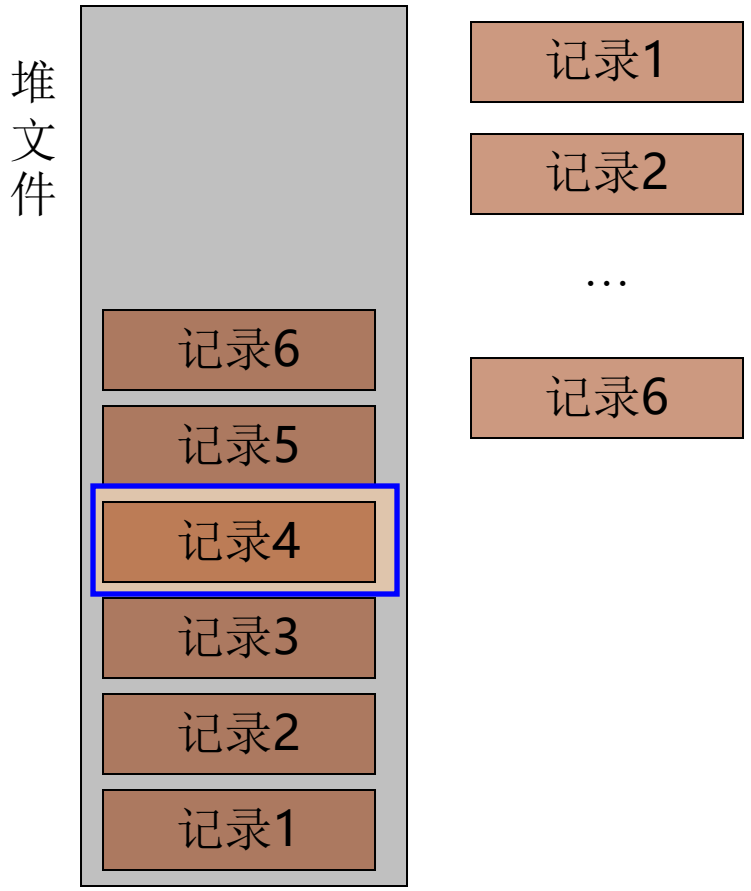
Requirements of DBMS cont.

- Traditional file systems serve specific purposes, have single uses, and low levels of sharing; in contrast, files in a database are shared by all users, with some uses even being unknown.
- Reducing a DBMS‘s dependency on the OS increases its portability (可移植性).
- Once established, the data volume in traditional file systems tends to be stable, whereas the data volume in database files can vary significantly.

File Organization

- ***Heap file:*** records stored according to their inserted order, and retrieved sequentially. This is the most basic and general form of file organization.
- ***Direct file:*** the record address is mapped through hash function according to some attribute's value.
- ***Indexed file:*** index + heap file/cluster

Heap file



- 最简单、最早使用的文件结构。记录按其插入的先后次序存放（所有记录在物理上不一定相邻接）
- 堆文件插入容易、查找不方便（所提供的唯一存取路径就是顺序搜索）
- 适于访问全部或相当多的记录，对于其它类访问效率较低。（设文件记录数为N，查找某一记录的平均访问次数为N+1/2）
- 若堆文件的记录按检索属性排序，可用二分查找法。（但要以排序为代价）

Heap file

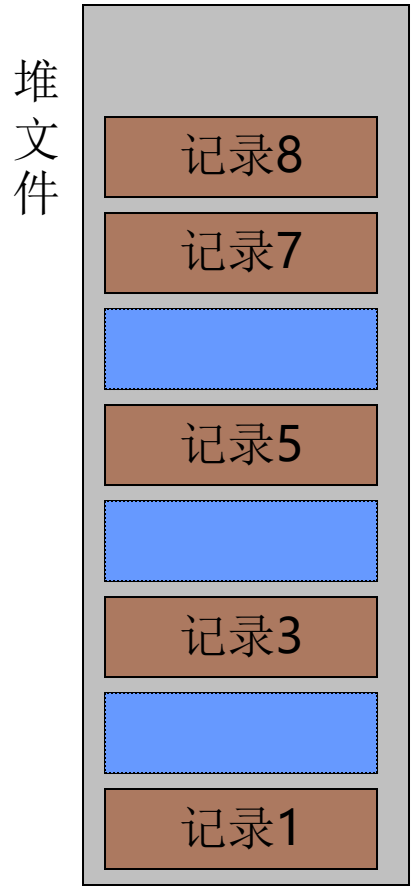
- **删除记录较麻烦**

假设，要删除右图堆文件中的第2，4，6条数据记录。

直接删除这三条记录的情况如右图所示。

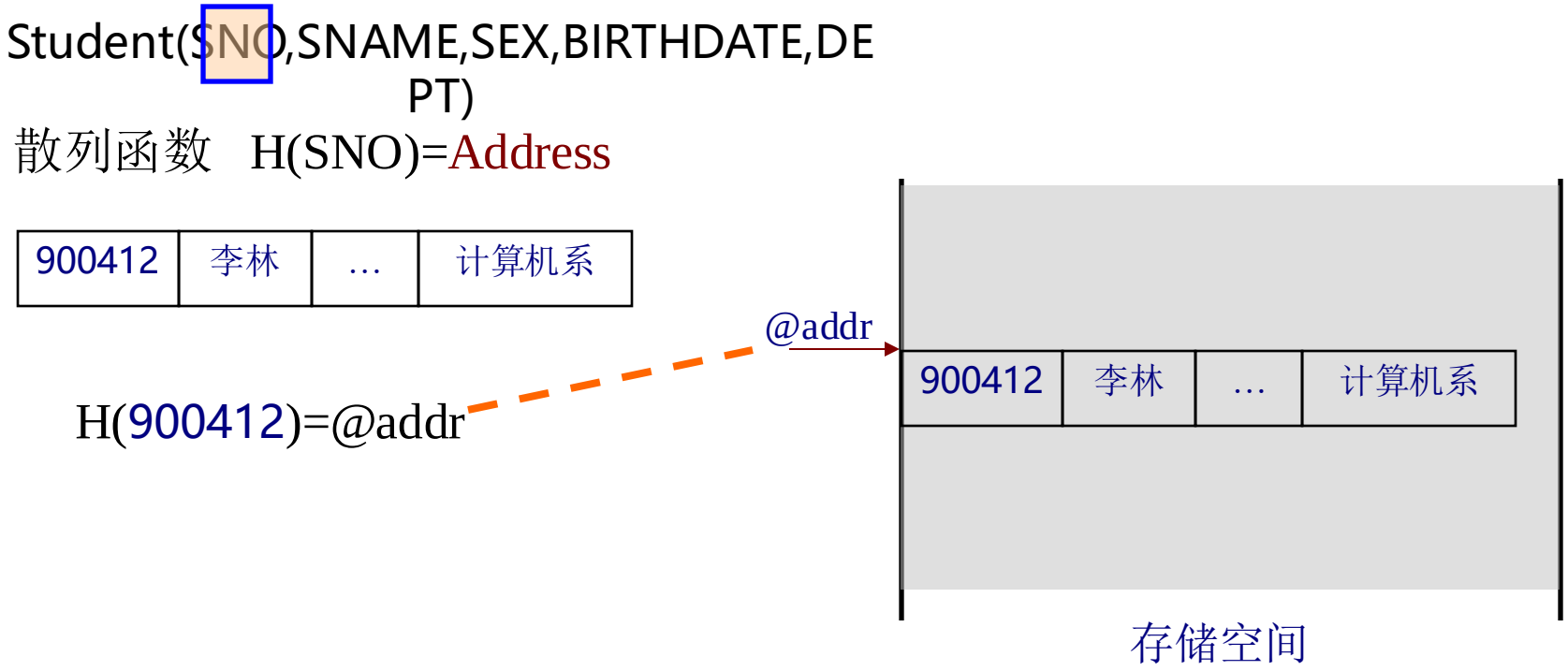
带来什么问题？

空间回收问题，后续记录需搬移，影响对文件的操作效率。



Direct file

- **直接文件中，将记录的某一属性用散列函数直接映射成记录的地址，被散列的属性称为散列键。**



Direct file

- 直接文件存在的问题：
  - 键所映射的地址范围固定（地址范围设的太大或太小都不好，为什么？）。
  - 地址重叠问题（处理地址重叠增加了开销）。
  - 直接文件只对散列键的访问有效。
  - 不便于处理变长记录。
  - 对于通用的DBMS很难找到通用的散列函数。
- 上述原因导致直接文件目前在数据库系统中使用不多。

Indexed file

- 索引相当于一个映射机构，把关系中相应记录的某个（些）属性的键值转换为该记录的存储地址（或地址集）。

存储空间

SNO	Address
900412	@addr2
900417	@addr1
900418	@addr3
…	…

学生表的索引文件

- 索引与散列的区别——索引文件有记录才占用存储空间，使用散列空文件也占用全部文件空间。
- 索引本省占用空间，但索引一般较记录小得多

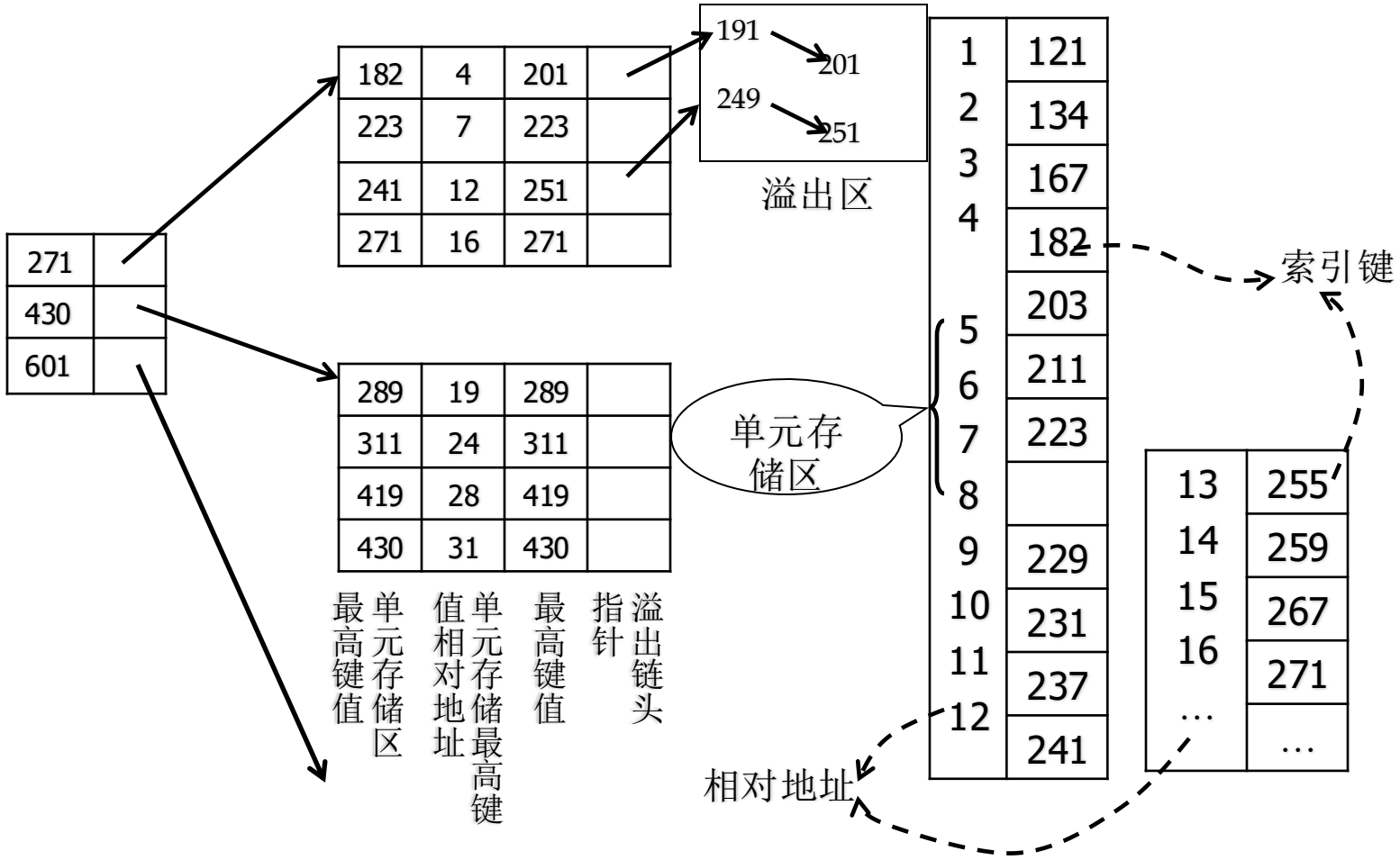
Indexed file

- 主索引——以主键为索引键(primary index)。
- 次索引——以非主键为索引键(secondary index)，建立次索引可以提高查询的效率，但增加了索引维护的开销。
- 倒排文件（inverted file）——主索引+次索引的极端情况（文件的所有属性上都建立索引），有利于多属性值的查找，但数据更新时开销很大。

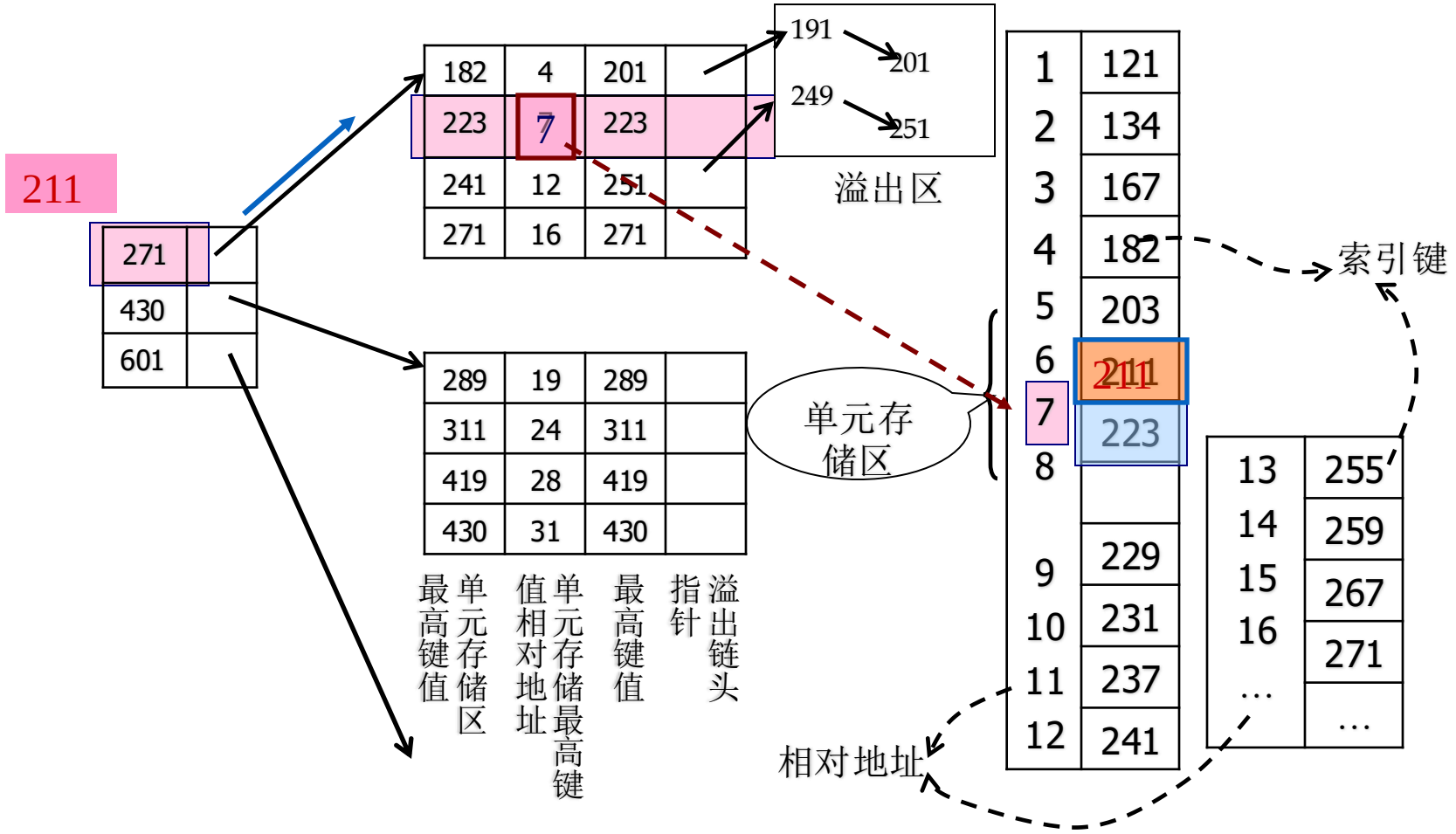
非稠密索引与稠密索引

- 不为每个键值设立索引项的索引——非稠密索引
- 可以节省索引的存储空间，代价是文件要按索引键排序
- 对一个文件，只能为一个索引键（一般为主键）建立非稠密索引（*为什么？*）
- 非稠密索引中，若干个记录组成一个单元存储区，单元存储区中的记录按索引键排序。
- 单元存储区装满后，可向溢出区溢出（用指针指向溢出区），但溢出太多时，指针链接次数增加，将导致数据库性能下降。
- 可以建立多级非稠密索引（通常最高一级索引应尽量保证可以常驻内存）。

非稠密索引示例


271
430
601
182 4 201
223 7 223
241 12 251
271 16 271
191 201
249 251
溢出区
289 19 289
311 24 311
419 28 419
430 31 430
单元存储区
最高键值
单元存储区
值相对地址
单元存储最高键
最高键值
溢出链头
指针
1 121
2 134
3 167
4 182
203
5
6 211
7 223
8
9 229
10 231
11 237
12 241
索引键
13 255
14 259
15 267
16 271
相对地址

查找索引键为211的记录的存储地址

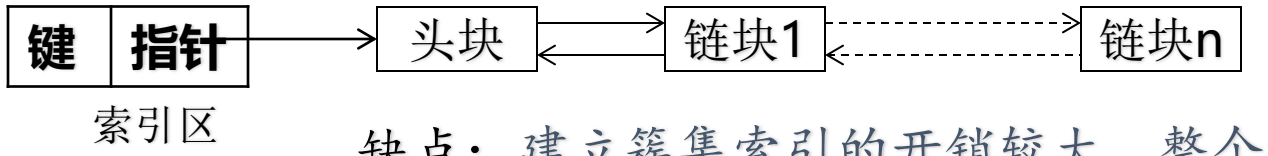


稠密索引（dense index）

- **每个键值对应一个索引项——稠密索引。**
- **稠密索引的预查找功能（用记录的地址代替记录参与集合运算，减少I/O次数）。**
- **索引溢出的问题**
- 稠密索引中，每增加一个键值，就要增加一个索引项。索引也会有溢出的可能。

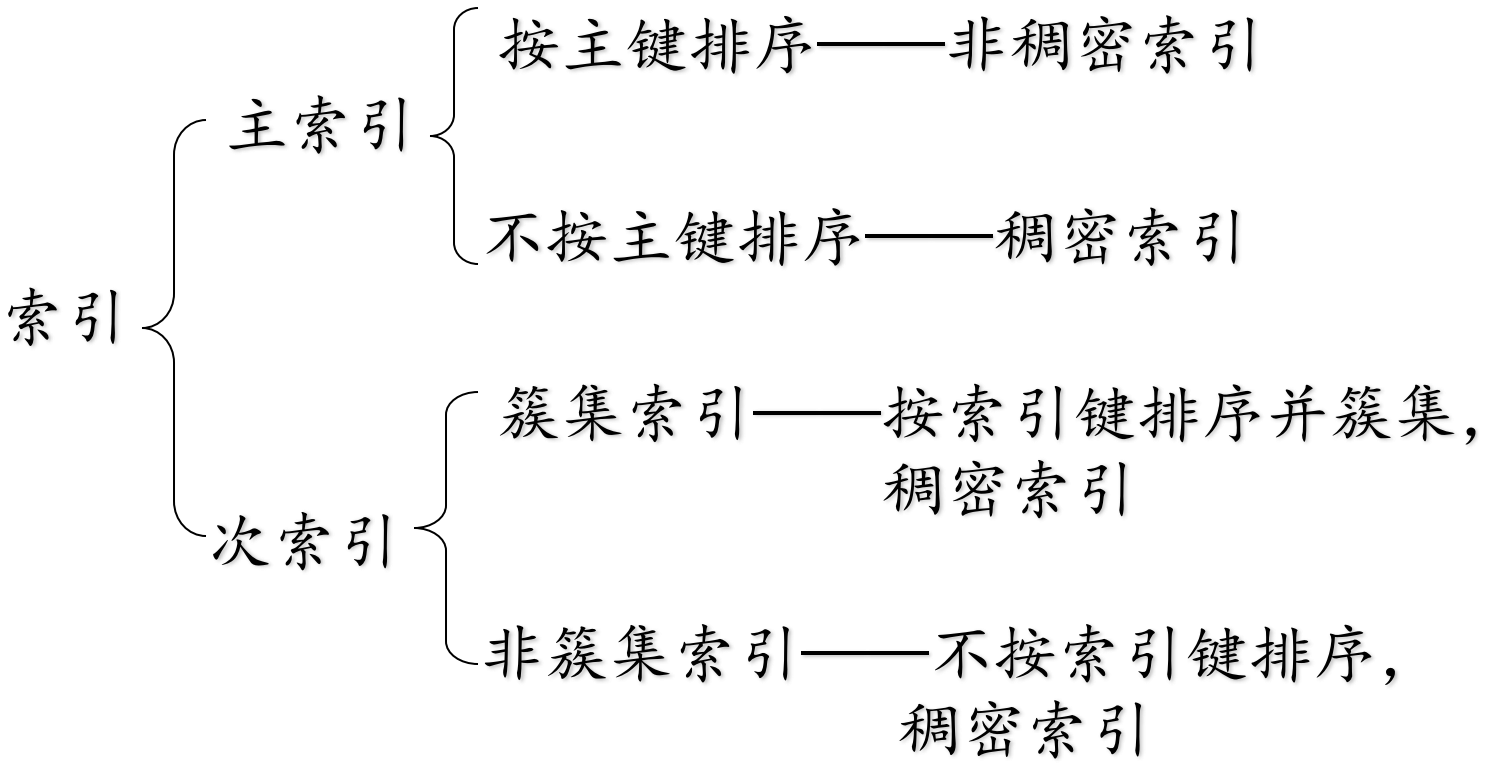
- 解决方法：1. 在每个索引块中预留发展的空间
- 2. 建立索引溢出区

- **稠密索引中，若键值不唯一，则在最低级索引中，每个键值对应的可能是一个地址集（对应多个记录）。如果这些记录分散在不同的物理块内，稠密索引的优点并不能体现出来（并不一定能减少I/O）。**
- 例如：某个键值对应100个记录，且这100个记录分散在100个物理块中，虽然通过稠密索引可以一次获得这100个地址，但仍然要访问100个物理块。
- 解决方法——簇集索引（clustering index）
  - 采用簇集索引：把键值相同的记录在物理地址上集中存放。



缺点：建立簇集索引的开销较大，整个文件要重新组织。

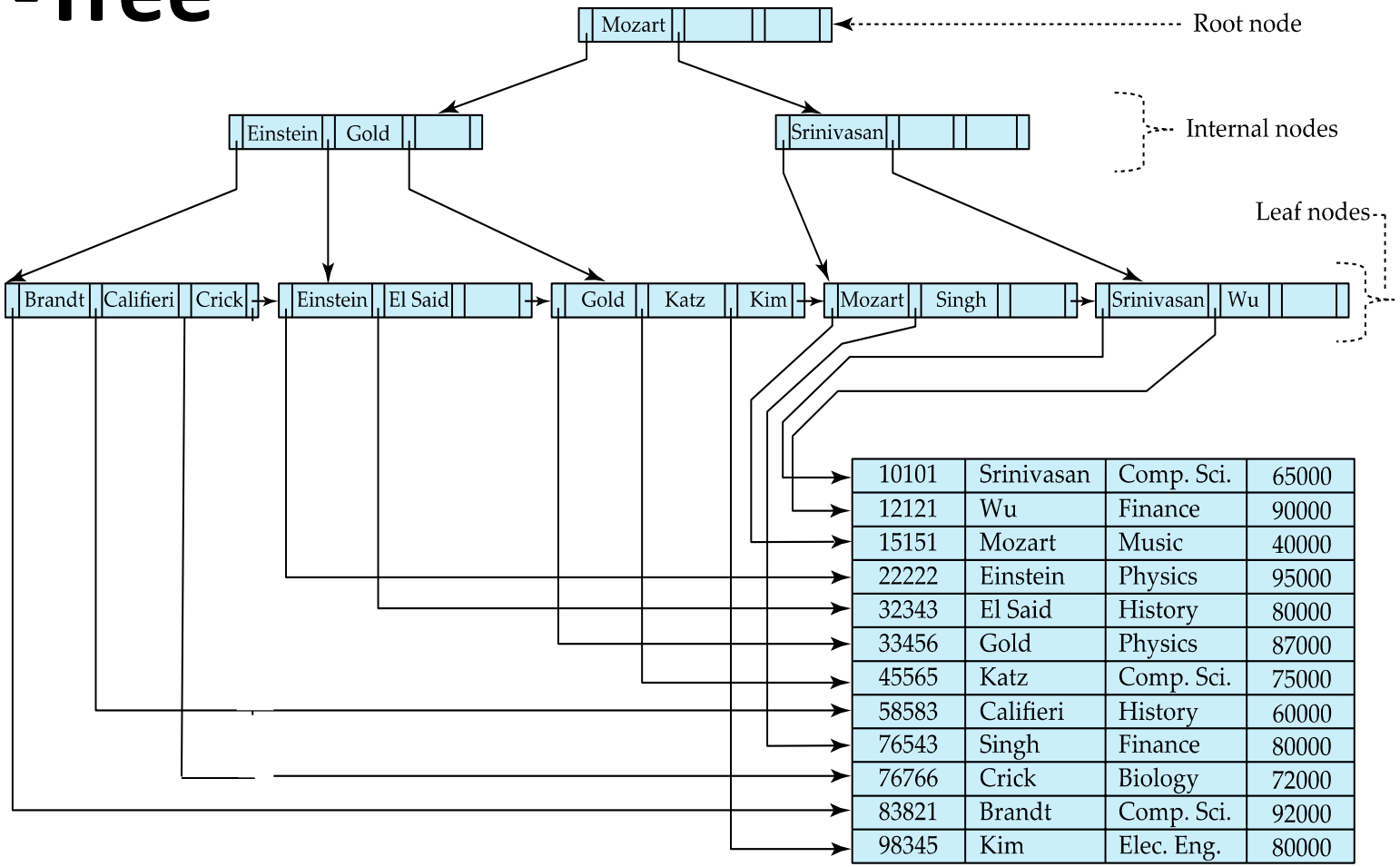
Summary of indexed file



Dynamic index

- **从数据结构角度看：静态索引是个多分树；动态索引是一种平衡多分树（即B-树），常用B^+树。**
- B^+树的限制和规定：
  - 每个节点最多有2k个键值，k称为B^+树的秩(Order)；
  - 根节点至少有一个键值，其它节点至少有k个键值，节点内，键值有序存放；
  - 除叶节点无子女外，其它节点若有J个键值，则有J+1个子女；
  - 所有叶节点都处于树的同一层，即树始终保持平衡。

Example of B+-Tree



10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	80000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	60000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

- ***Dynamic Tree Index***
  - ***Always Balanced***
  - ***Support efficient insertion & deletion***
  - ***Grows at root not leaves!***

B+-Tree Node Structure

- Typical node

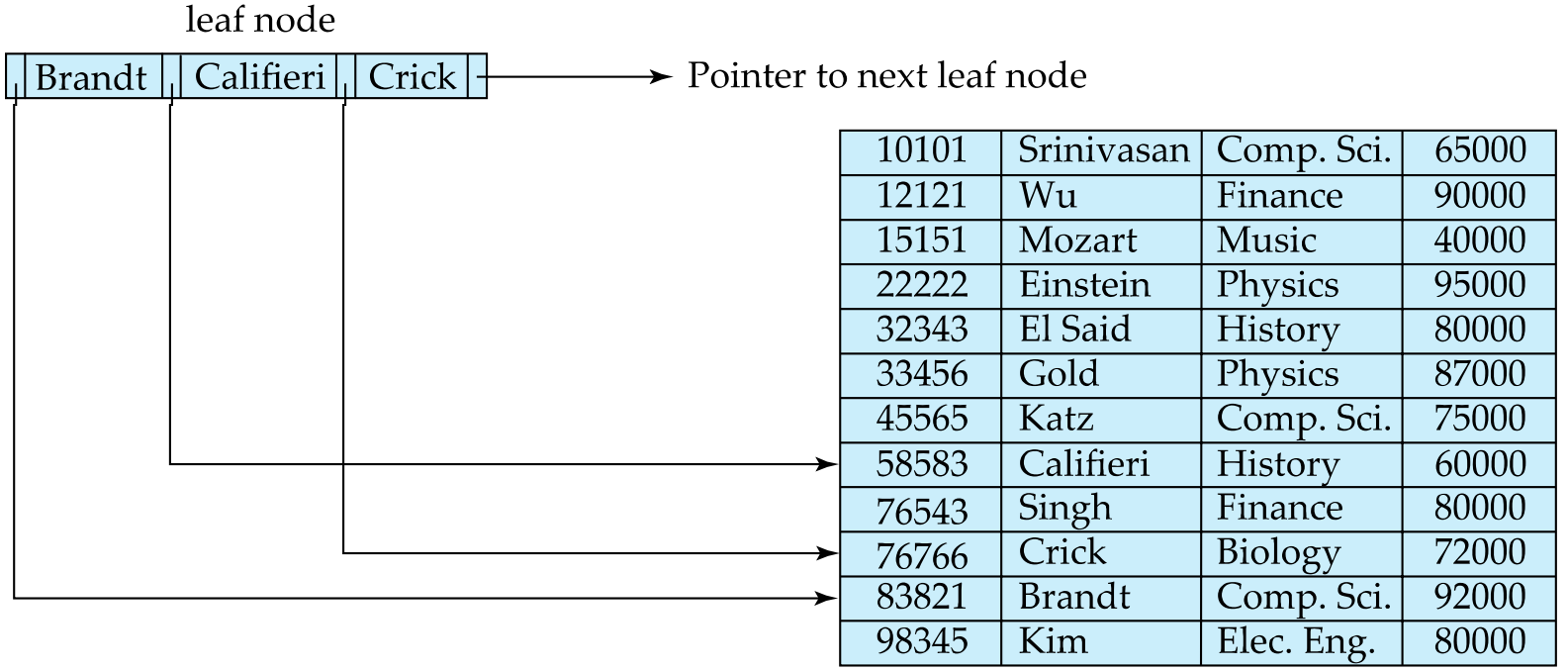
  - K_i are the search-key values
  - P_i are pointers to children (for non-leaf nodes) or pointers to records or buckets of records (for leaf nodes).
- The search-keys in a node are ordered

$$K_1 < K_2 < K_3 < \ldots < K_{n-1}$$

(Initially assume no duplicate keys, address duplicates later)

Leaf Nodes in B+-Trees

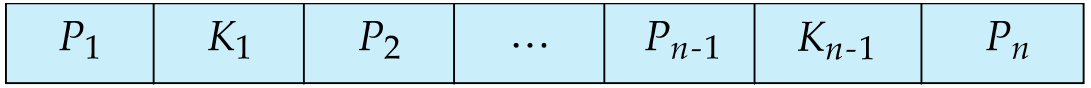
- For $i = 1, 2, \ldots, n-1$, pointer P_i points to a file record with search-key value K_i,
- If L_i, L_j are leaf nodes and $i < j$, L_i's search-key values are less than or equal to L_j's search-key values
- P_n points to next leaf node in search-key order



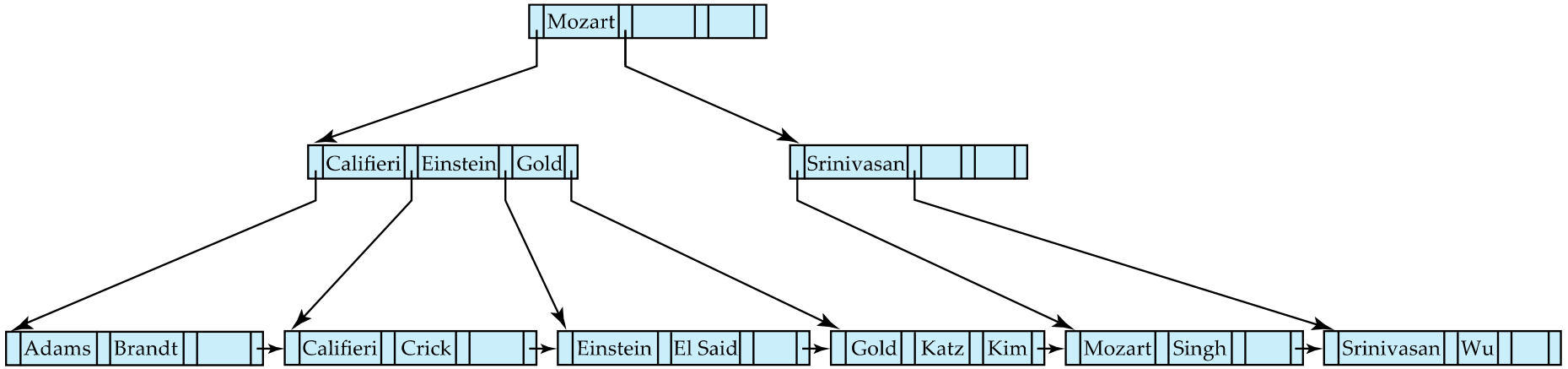
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	80000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	60000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Non-Leaf Nodes in B$^+$-Trees

- Non leaf nodes form a multi-level sparse index on the leaf nodes. For a non-leaf node with *m* pointers:
  - All the search-keys in the subtree to which P_1 points are less than K_1
  - For $2 \leq i \leq n - 1$, all the search-keys in the subtree to which P_i points have values greater than or equal to K_{i-1} and less than K_i
  - All the search-keys in the subtree to which P_n points have values greater than or equal to K_{n-1}
  - General structure

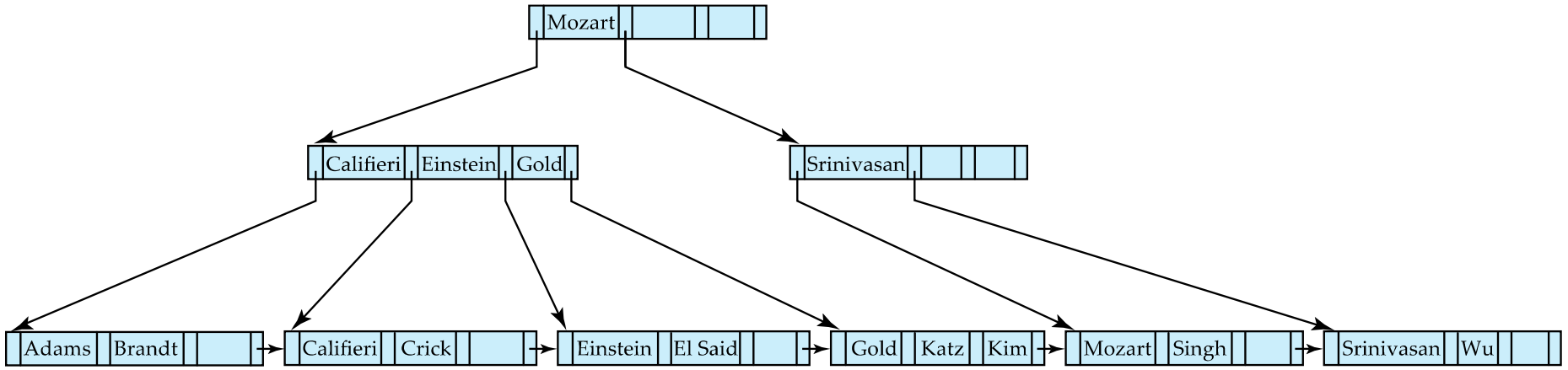


Queries on B+-Trees



Queries on B+-Trees (Cont.)

- **Range queries** find all records with search key values in a given range
  - Real implementations usually provide an iterator interface to fetch matching records one at a time, using a *next*() function



Queries on B+-Trees (Cont.)

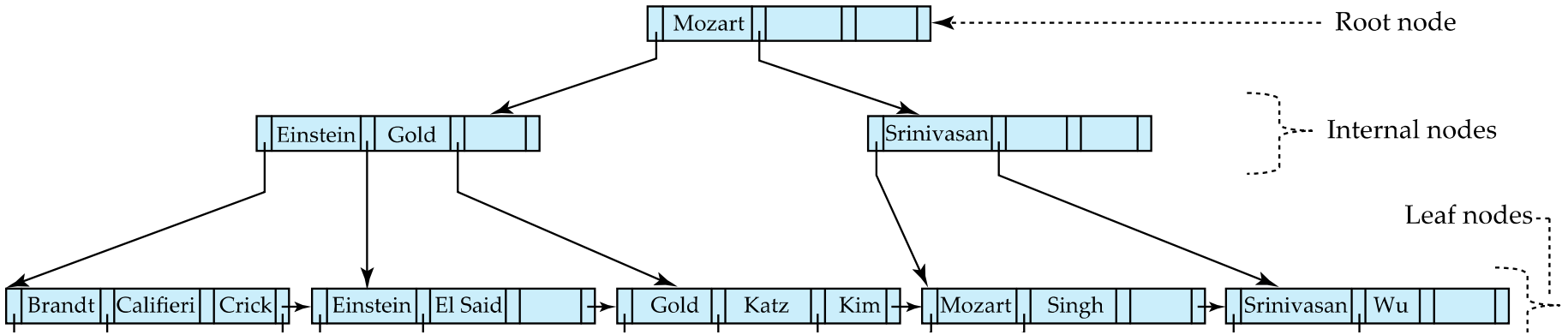
- **搜索B+树所需的I/O次数决定于其级数。**
- 设索引属性不同键值的数目为*N*，若索引键不是候选键，则记录数通常大于*N*。
- *B+*树的级数决定于*N*，而不是记录数！
- 假设*B+*树索引部分共有*L*级，其秩为*k*，则各级的最小结点数依次为：1，2，$2(k+1)$，…，$2(k+1)^{L-2}$
- 顺序集至少有$2(k+1)^{L-2}$个结点，得出：

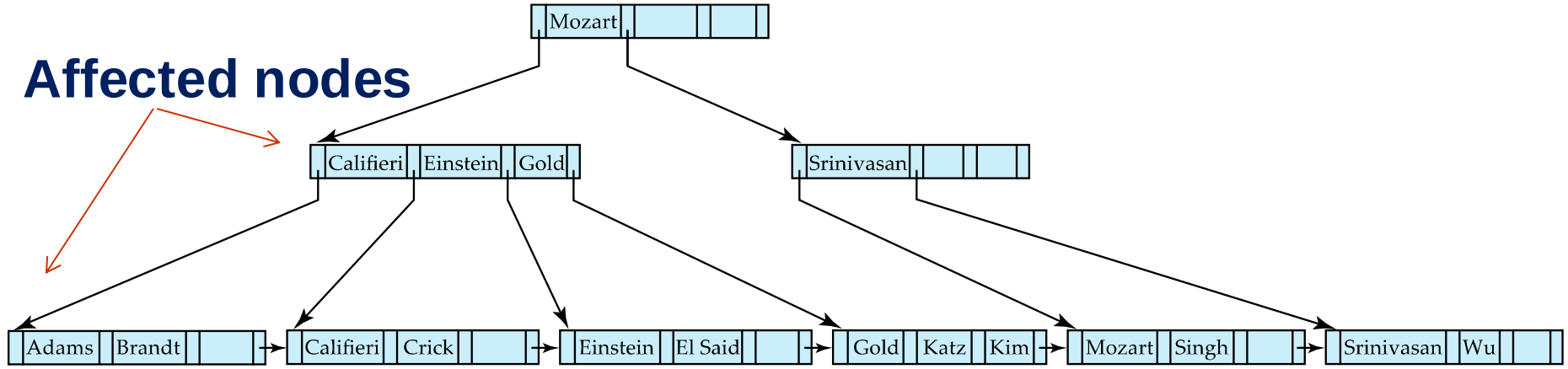
$$N \geq 2(k+1)^{L-2} \times k$$

$$L \leq 2+log_{(k+1)}(N/2k) \approx 1+ log_{(k+1)}(N/2)$$

- L决定了找到所需顺序集结点所需的I/O次数（访问数据还要额外的I/O），例如k=99，N=2,000,000，有L<5，即至多经过4次I/O就可以找到相应的顺序集结点。

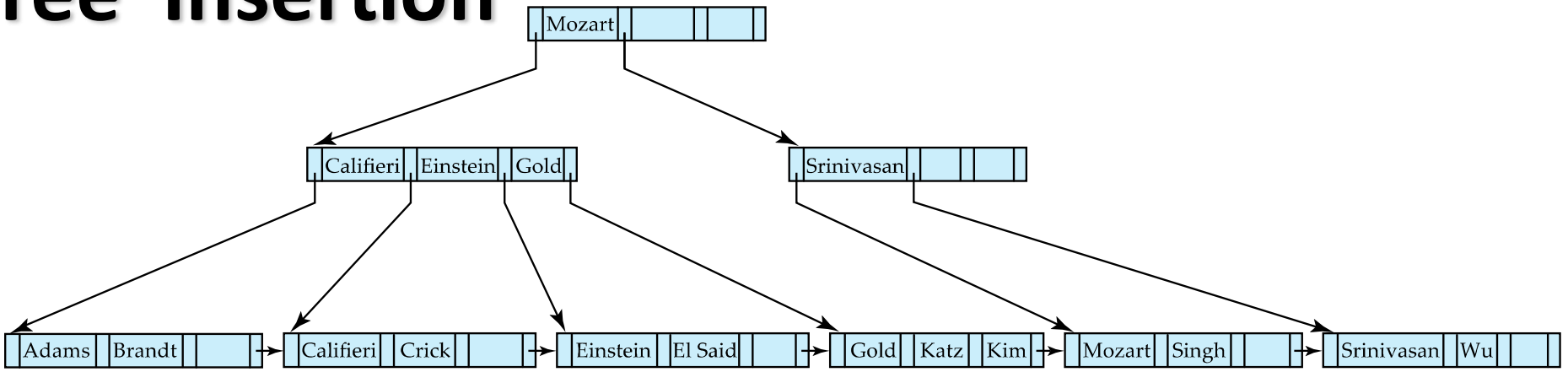
B+-Tree Insertion





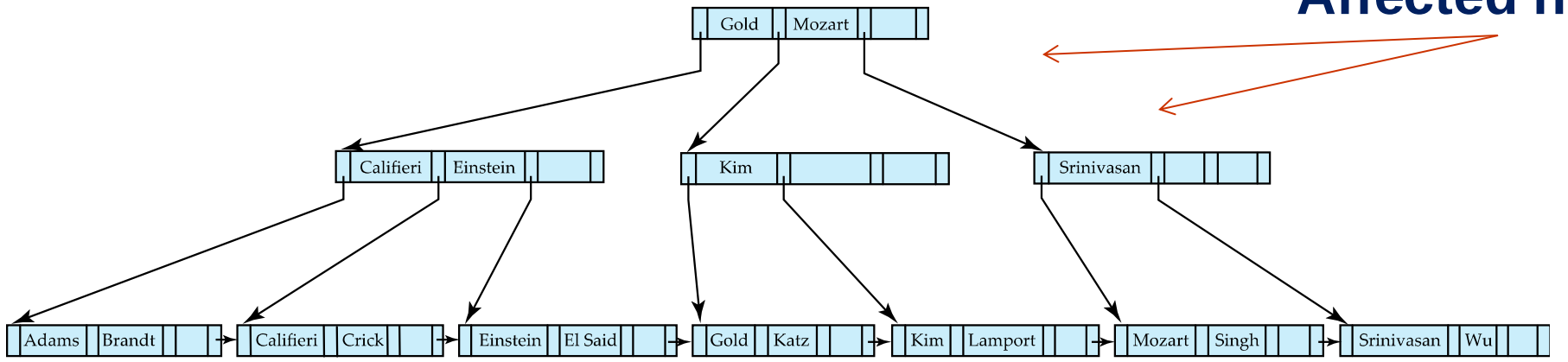
B+-Tree before and after insertion of “Adams”

B+-Tree Insertion



B+-Tree before and after insertion of "Lamport"

Affected nodes

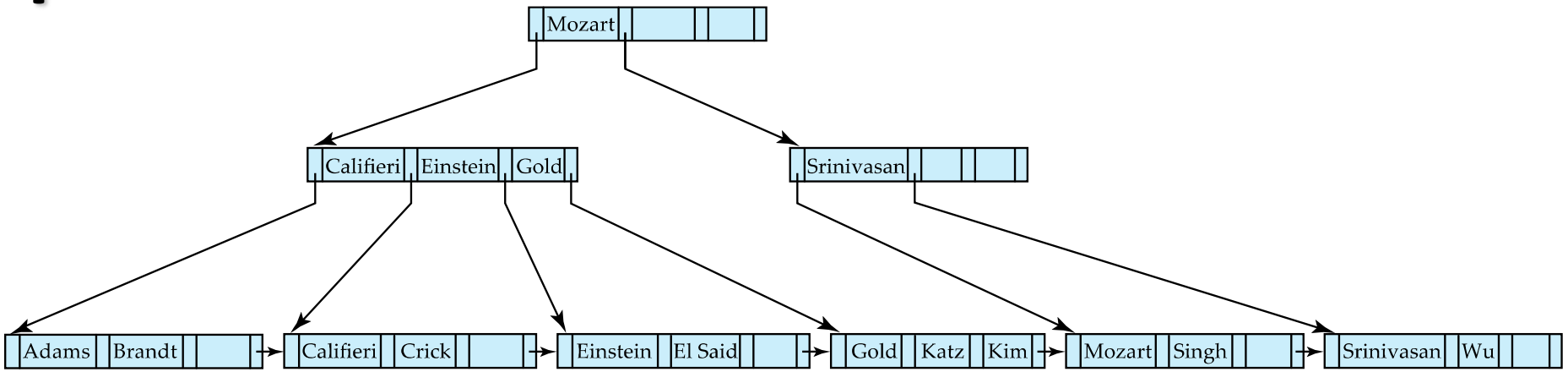


Affected nodes

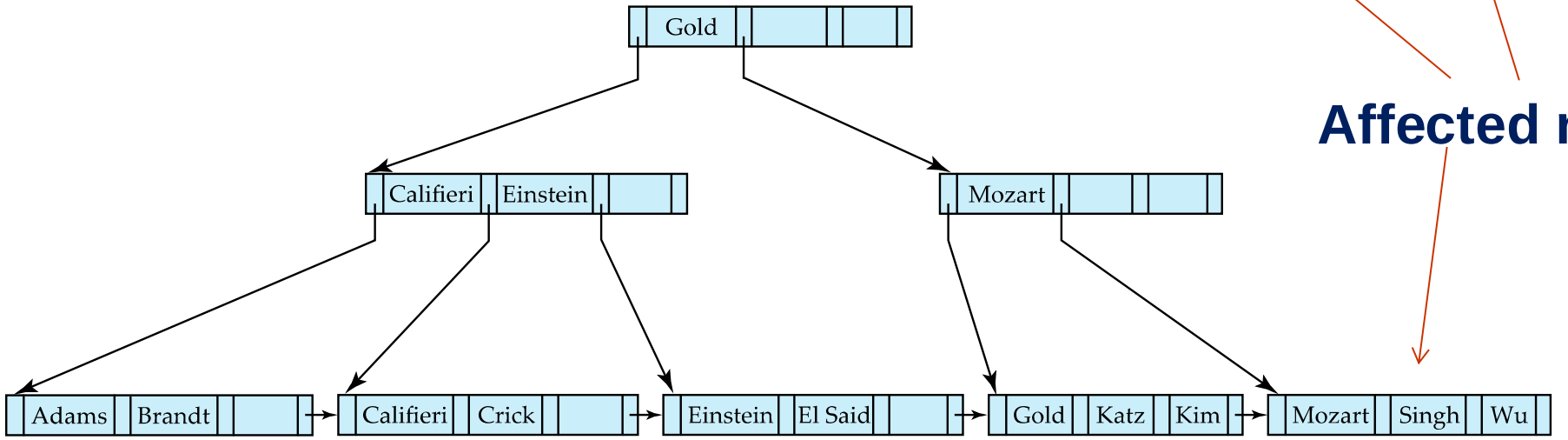
Insertion in B$^+$-Trees (Cont.)

- 从根向叶搜索，直至相应叶节点，若该叶节点不满，则将键值插入叶节点中；如叶节点已满（即已经有2K个键值），则将此叶节点分裂为二，叶节点分裂后，其双亲节点也必须增加一个键值，若双亲节点不满，插入过程结束；否则双亲节点继续分裂为两个节点，如此继续直到插入过程中止。

Examples of B+-Tree Deletion



Before and after deleting "Srinivasan"



Affected nodes

- Deleting "Srinivasan" causes **merging** of under-full leaves

Updates on B$^+$-Trees: Deletion

- 先从根节点出发，找到待删除键值所在叶节点；若删除该键值后，叶节点中键值数减为K−1个，则向其左右兄弟叶节点借一个键值，以保持每个叶节点存放键值不少于K个；若其左右叶节点都只有K个键值，则可将该叶节点与其左（或右）叶节点合并成包含2K−1个键值的叶节点，合并后，其双亲节点要减少一个键值，有可能导致双亲节点的合并。

B^+树提供3种存取路径：

1. 通过索引集进行树形搜索
2. 通过顺序集进行顺序搜索
3. 先通过索引找到入口，再沿顺序集顺序搜索

B+- Tree for secondary index

- B^+树实现的主索引稍加修改后也可用于次索引（把顺序集结点的tid换成指针，因为一个键值可能对应多个tid）。
- B^+树实现的各种索引都是稠密索引（非稠密索引的概念源于静态索引），提供了顺序搜索的功能，这是它的优点。

Other Index Technique*

- Dynamic hashing
- Grid structure file and partitioned hash function
- Bitmap index (used in data warehouse)
- Others